



**CEVA XM6™**

# **Vec-C programming reference guide**

**Rev. <1.0>**

**<Jun 2017>**



## Documentation Control

### *History Table*

| Version | Date     | Description   | Remarks |
|---------|----------|---------------|---------|
| 1.0     | Jun 2017 | • First draft |         |

## Disclaimer and Proprietary Information Notice

The information contained in this document is subject to change without notice and does not represent a commitment on any part of CEVA®, Inc. CEVA®, Inc. and its subsidiaries make no warranty of any kind with regard to this material, including, but not limited to implied warranties of merchantability and fitness for a particular purpose whether arising out of law, custom, conduct or otherwise.

While the information contained herein is assumed to be accurate, CEVA®, Inc. assumes no responsibility for any errors or omissions contained herein, and assumes no liability for special, direct, indirect or consequential damage, losses, costs, charges, claims, demands, fees or expenses, of any nature or kind, which are incurred in connection with the furnishing, performance or use of this material.

This document contains proprietary information, which is protected by U.S. and international copyright laws. All rights reserved. No part of this document may be reproduced, photocopied, or translated into another language without the prior written consent of CEVA®, Inc.

CEVA®, CEVA-XC™, CEVA-XC5™, CEVA-XC321™, CEVA-XC323™, CEVA-Xtend™, CEVA-XC4000™, CEVA-XC4100™, CEVA-XC4200™, CEVA-XC4210™, CEVA-XC4400™, CEVA-XC4410™, CEVA-XC4500™, CEVA-XC12™, CEVA-TeakLite™, CEVA-TeakLite-II™, CEVA-TeakLite-III™, CEVA-TL3210™, CEVA-TL3211™, CEVA-TeakLite-4™, CEVA-TL410™, CEVA-TL411™, CEVA-TL420™, CEVA-TL421™, CEVA-Quark™, CEVA-Teak™, CEVA-X™, CEVA-X1620™, CEVA-X1622™, CEVA-X1641™, CEVA-X1643™, Xpert-TeakLite-II™, Xpert-Teak™, CEVA-XS1100A™, CEVA-XS1200™, CEVA-XS1200A™, CEVA-TLS100™, Mobile-Media™, CEVA-MM1000™, CEVA-MM2000™, CEVA-SPT™, CEVA-VP™, CEVA-MM3000™, CEVA-MM3100™, CEVA-MM3101™, CEVA-XM™, CEVA-XM4™, CEVA-XM6™, CEVA-X1™, CEVA-X2™, CEVA-Audio™, CEVA-HD-Audio™, CEVA-VoP™, CEVA-Bluetooth™, CEVA-SATA™, CEVA-SAS™, CEVA-Toolbox™, SmartNcode™ are trademarks of CEVA, Inc.

All other product names are trademarks or registered trademarks of their respective owners.

## Support

CEVA® makes great efforts to provide a user-friendly software and hardware development environment. Along with this, CEVA provides comprehensive documentation, enabling users to learn and develop applications on their own. Due to the complexities involved in the development of DSP applications that might be beyond the scope of the documentation, an online Technical Support Service has been established. This service includes useful tips and provides fast and efficient help, assisting users to quickly resolve development problems.

### How to Get Technical Support:

- **FAQs:** Visit our website <http://www.ceva-dsp.com> or your company's protected page on the CEVA website for the latest answers to frequently asked questions.
- **Application Notes:** Visit our website <http://www.ceva-dsp.com> or your company's protected page on the CEVA website for the latest application notes.
- **Email:** Use the CEVA central support email address [ceva-support@ceva-dsp.com](mailto:ceva-support@ceva-dsp.com). Your email will be forwarded automatically to the relevant support engineers and tools developers who will provide you with the most professional support to help you resolve any problem.
- **License Keys:** Refer any license key requests or problems to [sdtkeys@ceva-dsp.com](mailto:sdtkeys@ceva-dsp.com). For SDT license keys installation information, see the *SDT Installation and Licensing Scheme Guide*.

**Email:** [ceva-support@ceva-dsp.com](mailto:ceva-support@ceva-dsp.com)

**Visit us at:** [www.ceva-dsp.com](http://www.ceva-dsp.com)

## List of Sales and Support Centers

| Israel                                                                                                                                                                                        | USA                                                                                                                                                                                          | Ireland                                                                                                                                                                                       | Sweden                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 Maskit Street<br>P.O. Box 2068<br>Herzelia 46120<br>Israel<br><br><b>Tel:</b> +972 9 961 3700<br><b>Fax:</b> +972 9 961 3800                                                                | 1174 Castro Street<br>Suite 210<br>Mountain View, CA<br>94040<br>USA<br><br><b>Tel:</b> +1-650-417-7923<br><b>Fax:</b> +1-650-417-7924                                                       | 18/19 South William<br>Street<br>2 <sup>nd</sup> Floor<br>Dublin 2<br>Ireland<br><br><b>Tel:</b> +353 1 237 3900<br><b>Fax:</b> +353 1 237 3923                                               | Klarabergsviadukten<br>70 Box 70396 107 24<br>Stockholm<br>Sweden<br><br><b>Tel:</b> +46(0)8 506 362 24<br><b>Fax:</b> +46(0)8 506 362 20                                                              |
| China<br>(Shanghai)                                                                                                                                                                           | China<br>(Beijing)                                                                                                                                                                           | China<br>(Shenzhen)                                                                                                                                                                           | Hong Kong                                                                                                                                                                                              |
| Unit 1203, Building E<br>Chamtime Plaza Office<br>Lane 2889, Jinke Road<br>Pudong New District<br>Shanghai, 201203<br>China<br><br><b>Tel:</b> +86-21-20577000<br><b>Fax:</b> +86-21-20577111 | Rm 503, Tower C<br>Raycom InfoTech Park<br>No. 2, Kexueyuan South<br>Road<br>Haidian District<br>Beijing 100190<br>China<br><br><b>Tel:</b> +86-10 5982 2285<br><b>Fax:</b> +86-10 5982 2284 | Rm 709, Tower A<br>SCC Financial Centre<br>No. 88 First Haide<br>Avenue<br>Nanshan District<br>Shenzhen 518064<br>China<br><br><b>Tel:</b> +86-755-8435 6038<br><b>Fax:</b> +86-755-8435 6077 | Level 43, AIA Tower<br>183 Electric Road<br>North Point<br>Hong Kong<br><br><b>Tel:</b> +852-39751264                                                                                                  |
| South Korea                                                                                                                                                                                   | Taiwan                                                                                                                                                                                       | Japan                                                                                                                                                                                         | France                                                                                                                                                                                                 |
| #478, Hyundai Arion<br>147, Gungok-Dong<br>Bundang-Gu<br>Sungnam-Si<br>Kyunggi-Do, 463-853<br>South Korea<br><br><b>Tel:</b> +82-31-704-4471<br><b>Fax:</b> +82-31-704-4479                   | Room 621<br>No. 1, Industry E, 2nd<br>Rd<br>Hsinchu, Science Park<br>Hsinchu 300<br>Taiwan R.O.C<br><br><b>Tel:</b> +886 3 5798750<br><b>Fax:</b> +886 3 5798750                             | 1-6-5 Shibuya<br>SK Aoyama Bldg. 3F<br>Shibuya-ku, Tokyo<br>150-0002<br>Japan<br><br><b>Tel:</b> +81-3-5774-8250                                                                              | RivieraWaves S.A.S<br>400, avenue Roumanille<br>Les Bureaux Green Side<br>5, Bât 6<br>06410 Biot - Sophia<br>Antipolis<br>France<br><br><b>Tel:</b> +33 4 83 76 06 00<br><b>Fax:</b> +33 4 83 76 06 01 |

## Table of Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1. INTRODUCTION .....</b>                                     | <b>1</b>  |
| 1.1 Scope .....                                                  | 1         |
| 1.2 Audience .....                                               | 1         |
| <b>2. XM6 ARCHITECTURE FOR SOFTWARE DEVELOPERS .....</b>         | <b>3</b>  |
| 2.1 Architecture Overview .....                                  | 3         |
| 2.2 Block Diagram .....                                          | 4         |
| 2.3 Data Memory .....                                            | 5         |
| 2.4 Programmable Units .....                                     | 6         |
| 2.4.1 VPU .....                                                  | 6         |
| 2.4.2 LSU .....                                                  | 7         |
| 2.4.3 SPU .....                                                  | 9         |
| 2.5 DMA and System Considerations .....                          | 10        |
| 2.5.1 TRANSFER DIRECTIONS: .....                                 | 10        |
| 2.5.2 CONTENTION-LESS IDM ACCESS .....                           | 11        |
| 2.5.3 DMA QUEUE .....                                            | 11        |
| 2.5.4 AUTOMATIC EXTERNAL PORT SELECTION .....                    | 11        |
| 2.5.5 CONFIGURABLE BURST TYPE .....                              | 11        |
| 2.5.6 CONFIGURABLE MAXIMUM EXTERNAL MASTER PORT BURST SIZE ..... | 11        |
| 2.5.7 BYTE-ALIGNED .....                                         | 11        |
| 2.5.8 CONFIGURABLE OUTSTANDING BURSTS .....                      | 11        |
| 2.5.9 TRANSFER ACCESS TYPES .....                                | 11        |
| <b>3. SIMD PROGRAMMING OVERVIEW .....</b>                        | <b>13</b> |
| <b>4. CEVA-XM6 DSP VEC-C .....</b>                               | <b>15</b> |
| 4.1 Vec-C Introduction .....                                     | 15        |
| 4.2 Data Types .....                                             | 15        |
| 4.3 Packing and Unpacking .....                                  | 17        |
| 4.4 Casting .....                                                | 18        |
| 4.5 Syntax of Instructions .....                                 | 19        |
| 4.6 Operators .....                                              | 21        |
| 4.7 Vector Predicated Execution .....                            | 21        |
| 4.8 Vselect() .....                                              | 22        |
| 4.8.1 VECTOR COMPARISON .....                                    | 22        |
| 4.8.2 REMARKS ON VSELECT() AND VPR USAGE .....                   | 23        |
| 4.8.3 SCALAR PREDICATION .....                                   | 24        |
| 4.9 Memory Access .....                                          | 24        |
| 4.9.1 ALIGNED AND NONE-ALIGNED VECTOR DATA ACCESS .....          | 25        |
| 4.9.2 PARALLEL DATA ACCESS (PROTECTED AND UNPROTECTED) .....     | 26        |
| 4.9.3 HISTOGRAM - VHIST .....                                    | 27        |
| 4.9.4 IMPORTANT NOTES ON MEMORY USAGE .....                      | 28        |
| <b>5. INSTRUCTIONS OVERVIEW .....</b>                            | <b>30</b> |
| 5.1 Assignment and Comparing .....                               | 31        |

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| 5.2       | Permutation .....                           | 31        |
| 5.2.1     | VECTOR PERMUTATION .....                    | 31        |
| 5.2.2     | INTERLEAVING .....                          | 32        |
| 5.3       | Bit Manipulation .....                      | 32        |
| 5.4       | Arithmetic .....                            | 33        |
| 5.5       | Intra Operations .....                      | 35        |
| 5.6       | Convolution sliding-window .....            | 36        |
| 5.6.1     | CONVOLUTION OF AN ELEMENT .....             | 37        |
| 5.6.2     | DATA SLICES .....                           | 37        |
| 5.6.3     | MULTI FILTER .....                          | 38        |
| 5.6.4     | FILTER DIMENSIONS .....                     | 39        |
| 5.6.5     | FILTER PATTERN .....                        | 39        |
| 5.6.6     | STEPS .....                                 | 40        |
| 5.6.7     | CLIPPING .....                              | 40        |
| 5.6.8     | VSWGEMAC CONTROL WORDS .....                | 41        |
| 5.6.9     | IMPORTANT NOTES .....                       | 42        |
| 5.7       | Other Sliding-Window instructions .....     | 43        |
| 5.7.1     | SUM OF DIFFERENCE SLIDING WINDOW .....      | 43        |
| 5.7.2     | SLIDING WINDOW, SUBTRACT AND COMPARE .....  | 43        |
| 5.7.3     | SLIDING WINDOW, SUBTRACT AND SATURATE ..... | 44        |
| 5.8       | Special data access .....                   | 45        |
| 5.8.1     | CHECKERED LOAD/STORE .....                  | 45        |
| 5.8.2     | OVERLAP LOAD/STORE .....                    | 45        |
| 5.9       | Parallel data access .....                  | 46        |
| 5.10      | None-Linear math functions .....            | 47        |
| 5.10.1    | VINV – VECTOR INVERSION .....               | 47        |
| 5.10.2    | VSQRT – VECTOR SQUARE-ROOT .....            | 47        |
| 5.10.3    | VSQRTI – VECTOR INVERSE SQUARE-ROOT .....   | 48        |
| 5.10.4    | VATAN – VECTOR ARC-TANGENT .....            | 49        |
| 5.10.5    | VTANH – VECTOR HYPERBOLIC TANGENT .....     | 50        |
| 5.10.6    | VEXPI – NATURAL EXPONENT INVERTED .....     | 50        |
| 5.10.7    | VLOGE – NATURAL LOGARITHM .....             | 50        |
| 5.10.8    | VSIN – SINE AND COSINE .....                | 51        |
| 5.11      | floating-point .....                        | 51        |
| <b>6.</b> | <b>CODE OPTIMIZATION GUIDE .....</b>        | <b>52</b> |
| 6.1       | Porting to CEVA-XM work flow .....          | 52        |
| 6.2       | Reference Algorithm .....                   | 53        |
| 6.3       | Fixed point porting .....                   | 53        |
| 6.4       | Memory constraints considerations .....     | 54        |
| 6.5       | Multi-Scalar optimization technique .....   | 57        |
| 6.6       | Writing kernels in FIS .....                | 59        |
| 6.7       | Using CEVA-XM software platforms .....      | 61        |
| 6.7.1     | CEVA-CV .....                               | 61        |
| 6.7.2     | CEVA-VX .....                               | 63        |
| <b>7.</b> | <b>OPTIMIZED KERNELS OVERVIEW .....</b>     | <b>65</b> |
| 7.1       | Mean and Variance .....                     | 65        |

|                                     |    |
|-------------------------------------|----|
| 7.2 Separable Correlation 7x7 ..... | 68 |
|-------------------------------------|----|

## List of Code Snippets

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Code 4-1: code example to summerize array of integers .....           | 15 |
| Code 4-2: Variable declaration is using a simple C/C++ interface..... | 17 |
| Code 4-3: pack / unpack prototypes.....                               | 17 |
| Code 4-4: unpacking example .....                                     | 17 |
| Code 4-5: vector decleration .....                                    | 18 |
| Code 4-6: vector casting with saturation .....                        | 18 |
| code 4-7: Vec-C code, using operators.....                            | 21 |
| Code 4-8: vselect example .....                                       | 22 |
| Code 4-9 : if statement SIMD extended .....                           | 22 |
| Code 4-10: example of protected VPLD instruction usage .....          | 26 |
| Code 5-1:vintrasum() example .....                                    | 36 |
| Code 6-1:Sobel reference implementation .....                         | 54 |
| Code 6-2:Harris corener reference implementation.....                 | 54 |
| Code 6-3:Sobel Vec-C implementation .....                             | 59 |
| Code 6-4:Harris corner Vec-C implementation .....                     | 60 |
| Code 6-5:CEVA-VX example .....                                        | 63 |
| Code 7-1: IntensityStats2 Vec-C function.....                         | 66 |
| Code 7-2:The resulted assembly code (main loop only).....             | 66 |
| Code 7-3:Seperable7x7 correlation envelope .....                      | 68 |
| Code 7-4:Seperable 1x7 (horizontal) correlation kernel.....           | 69 |
| Code 7-5:Seperable 7x1 (vertical) correlation kernel .....            | 69 |

## List of Figures

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Figure 1 XM6 block diagram .....                                         | 4  |
| Figure 2: Memory structure .....                                         | 5  |
| Figure 4: Vector registers partitions.....                               | 16 |
| Figure 5: none-aligned data access .....                                 | 25 |
| Figure 6: parallel and relative-parallel access.....                     | 27 |
| Figure 7:interleaving instructions .....                                 | 32 |
| Figure 8: single slice 1D, vector-B is the continuation of vector-A..... | 37 |
| Figure 9: two slices with single filter (reuse).....                     | 37 |
| Figure 10: two slices, two filters, no filter reuse .....                | 38 |
| Figure 11: Multi-filters, single slice .....                             | 38 |
| Figure 12: Multi-filters, multi-slices .....                             | 38 |
| Figure 13: 2D filters .....                                              | 39 |
| Figure 14: sliding window pattern example .....                          | 40 |
| Figure 15: 2D filter, pattern=0xCC, skip=0 .....                         | 40 |
| Figure 16: 2D filter, pattern=0xCC, skip=1 .....                         | 40 |
| Figure 17: checkered load .....                                          | 45 |
| Figure 18: vector load overlap operation .....                           | 45 |

|                                                            |    |
|------------------------------------------------------------|----|
| Figure 19: algorithm porting flow .....                    | 52 |
| Figure 20: Harris corner system flow .....                 | 56 |
| Figure 3: Multi-Scalar memory structure One-Bank .....     | 58 |
| Figure 3: Multi-Scalar memory structure Two-Bank .....     | 58 |
| Figure 21: CEVA-VX Harris corner flow .....                | 64 |
| Figure 22: The following diagram shows the data flow ..... | 68 |

## List of Tables

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Table 2-1: Vector Processing <-> Load/Store Latency Table .....      | 8  |
| Table 2: Vec-C data types .....                                      | 16 |
| Table 3: vcast options .....                                         | 18 |
| Table 4: Vec-C and Assembly syntax .....                             | 20 |
| Table 5: Examples for instructions with multiple return-values ..... | 20 |
| Table 6: Vec-C overloaded operators .....                            | 21 |
| Table 7: VPR related latency .....                                   | 23 |
| Table 8: CEVA-XM6 memory access bandwidth .....                      | 25 |
| Table 9: relative and absolute address calculations .....            | 27 |
| Table 10: instructions glossary .....                                | 30 |
| Table 11: bit manipulation instructions .....                        | 32 |
| Table 12: Arithmetic instructions index .....                        | 33 |
| Table 13: glossary .....                                             | 35 |
| Table 14: intra instructions .....                                   | 36 |
| Table 5-15: Vswgenmac rE Register Register Field Description .....   | 41 |
| Table 5-16: VSWGENMAC rF Register Field Description .....            | 41 |
| Table 17: Parallel load/store instructions .....                     | 46 |
| Table 18: Harris corner theoretical buffers .....                    | 56 |
| Table 19: Harris corner runtime buffers .....                        | 57 |
| Table 20: CEVA-CV kernels .....                                      | 62 |
| Table 21: CEVA-CV algorithms .....                                   | 62 |

# **1. Introduction**

## **1.1 Scope**

This is a programming guide describing the Vec-C programming extension for CEVA-XM6 DSP platform.

## **1.2 Audience**

New and experienced software developers working on CEVA-XM6 core.



## 2. XM6 Architecture for Software Developers

### 2.1 Architecture Overview

The CEVA-XM6 is a DSP core targeted for high-performance computer vision and image-processing applications for low power applications.

XM6 architecture combines Very-Long-Instruction-Word (VLIW) and Single-Instruction/Multiple-Data (SIMD), with two Load/Store units, 3 vector processing units and 4 scalar units the XM6 achieves very high data throughput with low power

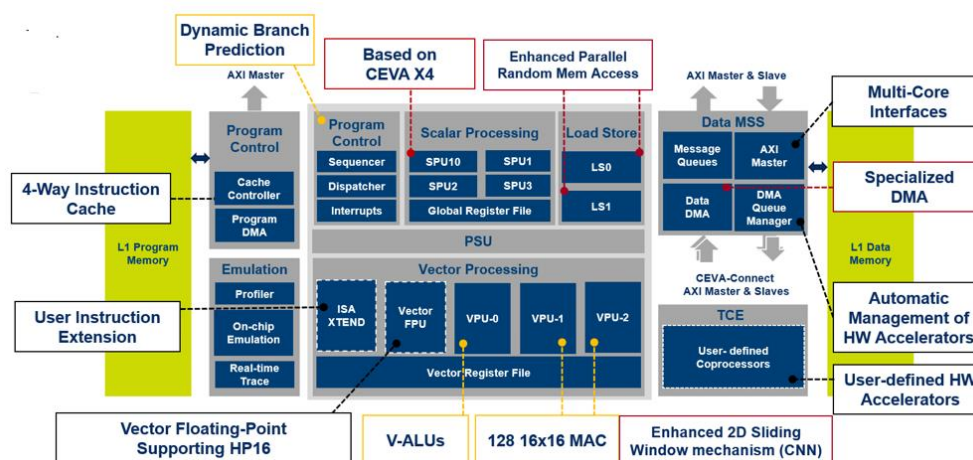
Summary of XM6 features:

- 64 32-bit general registers for scalar operations and address generation
- 48 256-bit vector registers used for SIMD processing
- Two Vector Processing Units (VPUs):
  - ALU Instructions that supports all fixed-point and complex data types operations in SIMD processing
  - Data permutation, logic operations, none-linear arithmetic, bit counting, saturation, Hamming, type conversion and other
  - Floating-point IEEE-754 SIMD processing (support for half-precision as well)
  - Wide range of multiplication with (or without) accumulation
  - Sliding-Window instructions for convolutions, SAD, compare and subtraction
  - None linear math operations:  $\sin(x)$ ,  $\cos(x)$ ,  $\atan(x)$ ,  $\tanh(x)$ ,  $\exp(-x)$ ,  $1/\sqrt{x}$ ,  $1/x$ ,  $\sqrt{x}$
- One Light Vector Processing Unit (LVPU):
  - Instructions that supports all fixed-point data types ALU operations in SIMD processing
  - Data permutation, logic operations, none-linear arithmetic, bit counting, saturation, type conversion and other
- Four Scalar Processing Units (SPUs):
  - Supports for 16-bit, 32-bit and 64-bit operations
  - 16×16-bit, 32×16-bit, and 32×32-bit multipliers
  - 16×16-bit, 32×16-bit, and 32×32-bit MAC operations
  - Full support of bit-manipulation and logical unit

- Supports single-precision and half-precision floating-point operations
- Scalar registers (32-bit) SIMD instructions support
- Two Load/Store Units (LSUs) for two independent accesses to the data memories:
  - 512 bits bandwidth for load operations
  - 512 bits bandwidth for store operations
  - 32 different memory addresses generation in a single memory access
  - Multiple data addressing modes: direct, indirect, indexed, modulo, parallel and stack
  - Four gigabyte program and data address
  - Byte-addressable data address
  - None-aligned data memory access
- Branch Prediction Buffer (BTB)
- Histogram acceleration
- 4-way program cache
- Data DMA with queue management

## 2.2 Block Diagram

Figure 1 XM6 block diagram



## 2.3 Data Memory

CEVA-XM data Memory-Sub-System (MSS) has access to either internal Tightly Coupled Data Memory (IDM) or external memory via AXI bus. The IDM features are:

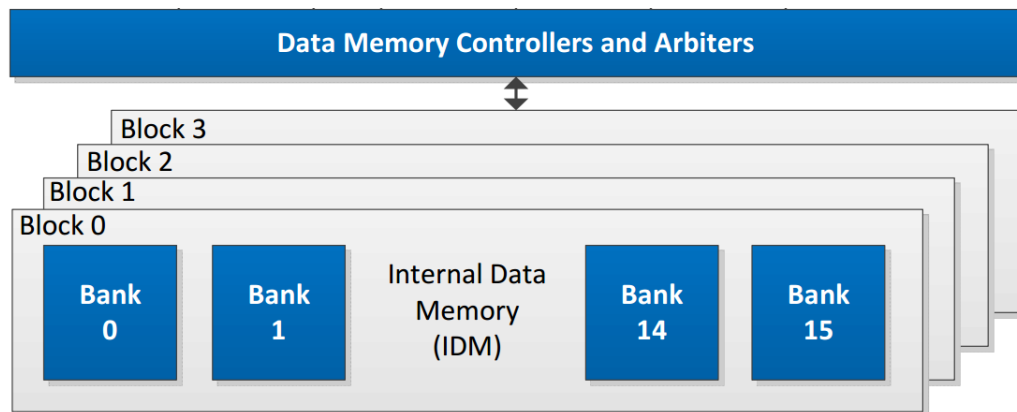
- Configurable size (HW) 256k / 512k / 1024k / 2048k
- Four memory blocks
- Each block is composed of 16 memory banks. The addresses are interleaved over the banks within each block, in the same row boundaries
- Bank size is 32-bit
- Memory data access is 512-bit (same as LSU0+LSU1 bandwidth)
- The total accessible address space is 32-bit (4GB)

This memory structure enables versatile access highly suitable for vision and image processing algorithms.

The IDM memory structure has crucial impact on algorithms and software performance. When used efficiently it can significantly affect the overall performance, incorrect design can have devastating impact ( [Memory constraints considerations](#) ).

The IDM memory structure of CEVA-XM cores is constructed of 4 memory blocks, each contains 16 memory banks. Access to IDM using LSU has zero penalty.

*Figure 2: Memory structure*



This unique memory structure enables both LSUs to access 32 different addresses each cycles, as long as the buffers do not cause bank or block conflicts.

## 2.4 Programmable Units

### 2.4.1 VPU

In the heart of CEVA-XM6 lay three Vector Processing Units that perform all the SIMD processing.

There are two fully quipped units (VPU0 and VPU1), capable of performing all processing operations including MAC (Multiply and Accumulate), filtering, permuting, intra-operations and arithmetic, and one unit (LVPU) which is capable of arithmetic and permutation operations.

#### 2.4.1.1 Vector Registers

The CEVA-XM6 contains a Vector Register File (VRF) that consists of 48 256-bit vector registers. The vector registers are used by the VPUs and LVPU for vector operations and as source and destination registers. They are used by the LSU for address pointers and for loading/storing data from/to the data memory. Pairs of vector registers can be coupled to form 512-bit accumulators.

Vectors are used by VPUs (and LSUs) as any of the following data-types:

| XM6 architecture types                                                | description                                                                                                       |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>vX.c32</code><br><code>vXh.c32</code> / <code>vXl.c32</code>    | 32 elements of 8-bit signed two's complement integer<br>high / low part of a vector, 16 elements of 8-bit         |
| <code>vX.uc32</code><br><code>vXh.uc32</code> / <code>vXl.uc32</code> | 32 elements of 8-bit unsigned integer<br>high / low part of a vector, 16 elements of 8-bit                        |
| <code>vX.s16</code><br><code>vXh.s16</code> / <code>vXl.s16</code>    | 16 elements of 16-bit signed two's complement integer<br>high / low part of a vector, 8 elements of 16-bit signed |
| <code>vX.us16</code><br><code>vXh.us16</code> / <code>vXl.us16</code> | 16 elements of 16-bit unsigned integer<br>high / low part of a vector, 8 elements of 16-bit unsigned              |
| <code>vX.i8</code>                                                    | 8 elements of 32-bit signed two's complement integer                                                              |
| <code>vX.ui8</code>                                                   | 8 elements of 32-bit unsigned integer                                                                             |
| <code>vX.l4</code>                                                    | 4 elements of 64-bit signed two's complement integer                                                              |
| <code>vX.u14</code>                                                   | 4 elements of 64-bit unsigned integer                                                                             |
| <code>vX.hf16</code>                                                  | 16 element of 16-bit floating point                                                                               |
| <code>vX.f8</code>                                                    | 8 elements of float                                                                                               |

#### 2.4.1.2 Vector Predicates

There are 31 VPR registers 32-bits each, each bit holds a value of true or false and controls the execution of its corresponding element in the SIMD instruction. The VPR registers are r33 through r63 and are part of the VRF registers, when no VPR is specified, then the execution is unconditional using dummy VPR constant of all ones.

More on VPR usage in [Vector Predicated execution](#) chapter.

## 2.4.2 LSU

The Load and Store Units (LSU) do the following:

- Address generation
- Data manipulations before store and after load
- Registers and Vector files read / write access

The LSU contains two independent load and store units:

- **LS0:** Executes both vector and scalar loads
- **LS1:** Executes vector loads and stores, as well as scalar stores

Scalar register file access bandwidth is 128 bits, which enables LS0 to read (and LS1 to write) only their maximum bandwidth with byte alignment.

The vector-register file can be accessed with 256-bits per LSU, which allows both the LS0 and LS1 to read their maximum bandwidth of 512 bits with byte-alignment address generation.

- The write data bandwidth is limited to 256 bits using only LS1, with byte-alignment address generation

All LSU instructions can be conditional using one of the predicate registers. The SPU contains dedicated instructions that can affect the predicate registers. In addition, vector load and store instructions can be predicated using the vector predicates to mask the load and store operations. This enables the LSU to load and store only part of the vector according to the vector predicates

### 2.4.2.1 Address Generation

Computes a single 32-bit address according to the addressing mode specified in the instruction or vector (up to 16 addresses when using parallel memory access). These addresses are then supplied to the data memory along with the read/write strobe and the access width. The address generation is done as follows:

- Linear address generation of a 32-bit address is done according to one of the addressing modes described in Section 7.4, [Addressing Modes](#). The address points to a byte location in the data memory space.
- Parallel address generation of eight or sixteen 32-bit addresses is done according to one of the parallel addressing modes. The addresses point to a byte location in the data memory space.

### 2.4.2.2 Data manipulation

LSU instructions can manipulate the read data in several manners

- Organizing it in various forms ([Special data access](#))
- Conditionally load or store data according to VPR value ( [Predicated execution](#) )
- Cast values and saturate them before storing it to memory ( [Special data access](#) )
- Pack / Unpack data before storing to memory

### 2.4.2.3 Write-Buffer

All store operations are written to the memory via Write-Buffer (WB) that queues the transactions .The WB queue reduces the number of core stall cycles caused by the write transactions (but does not eliminate all of them), by delaying the actual write transactions until the target can be written.

The transactions queue causes a delay in the write to memory process and reduces the core dependency on the memory availability.

Some hard to detect phenomena can be the result of the WB operation, for example in some kernels, which are store-bounded or perform bursts of store operations, the WB can be overflowed and produce core stalls. In other cases when bank contentions between delayed store operation and load operation might occur, in such case the WB will generate stall cycle that will be detected only in delay of several cycles.

### 2.4.2.4 LSU / VPU instructions latencies

The table below shows the latencies when accessing memory (LSU) and using vector (VPU) instructions.

**Table 2-1: Vector Processing <-> Load/Store Latency Table**

| First Instruction argument (write) | Second Instruction (read) |                 |                    |                     |
|------------------------------------|---------------------------|-----------------|--------------------|---------------------|
|                                    | VPU vA (data)             | VPU vacc (data) | VST/VPST vA (data) | VPST vN/ vM (index) |
| VPU early                          | 0                         | 0               | 0                  | 6(9)                |
| VPU middle                         | 5                         | 2               | 1                  | 7                   |
| VPU late                           | 2                         | 0               | 1                  | 9                   |
| VLD / VPLD                         | 2                         | 0               | 0                  | 6                   |

The latencies table shows the most important latencies limitations between two consecutive instructions. In the left column there are 3 possible VPU instruction arguments and one for VLSU:

- VPU early – relates to VPU instructions output arguments that are written in early stages of the pipeline. Marked in the specifications as vZ
- VPU middle – are VPU instructions output arguments that are written in middle of the pipeline. Marked in the specifications as vmZ
- VPU late – are VPU instructions output arguments that are written in late stages of the pipeline. Marked in the specifications as vrZ or vrW
- VLD / VPLD – are output arguments written by Load instructions

Each column represents a different type of input argument

- VPU vA – are arguments that are read by VPU instructions in the early stages of the pipeline, used by all instructions
- VPU vacc – are arguments that are read by VPU instructions in the late stages of the pipeline, usually used by MAC instructions.
- VST/VPST vA – are data arguments used by VLSU
- VPST rN/rM – are arguments that are used as index in VPLD (VBCPST, VSTMULTI) instructions, they are read in very early stages of the pipeline, therefore their latencies are the longest

### 2.4.3 SPU

The functional units perform independent operations in parallel, each according to the specific instruction within the instruction packet.

Each SPU instruction within the instruction packet can be independently conditional, based on one of 15 predicate registers.

For single instruction multiple data (SIMD) type instructions, each operation can be masked separately. The SPU incorporate three execution stages. As a result, various instructions take a different number of cycles to execute. The number of stages an instruction requires for execution determines the cycle penalty before the result can be used by another instruction. The stage count for each instruction is described in the Instruction Set in the *CEVA-XM6*

#### SPU Instructions

All instructions are identical for all units. The instructions can be either a single instruction or a SIMD-type instruction operating on two short integers simultaneously. The SPUs support the following types of operations:

- **Arithmetic Operations**

- **Logical Operations**
- **Bit-Manipulation Operations**
- **Miscellaneous Operations**
- **Multiply and Multiply-Accumulate Operations**

## **Supported Data Types**

The SPU supports the following data types:

- 8-bit character
- 16-bit short integer
- 32-bit integer
- 64-bit long integer
- 32-bit single precision floating point (compliant with IEEE-754)
- 16-bit half precision floating point (compliant with IEEE-754)

The integer data types have signed and unsigned variants.

A 16-bit short integer is supported in addition in a SIMD, where two short 16-bit values are stored in a single 32-bit register as a 2-element vector and can be processed in parallel by dedicated SIMD instructions.

The supported types of the sources and destinations are specifically described in each instruction in the *CEVA-XM6 Architecture Specification Volume II*.

- Programming SPU code is done mainly via C/C++ compiler (or assembly). Though there are many special instructions that do not have equivalent in C/C++, these instructions have Vec-C intrinsic functions.
- See in Vec-C navigator reference guide SPU section.

## **2.5 DMA and System Considerations**

The DMA transfers data between the external memory and the IDM or within the IDM. It accesses external memory via the EDP, AXIm0 and AXIm1 Master ports. The DDMA has the following capabilities:

### **2.5.1 Transfer Directions:**

- Moves data from the IDM to external memory.
- Moves data from external memory to the IDM.
- Moves data within the IDM.

## 2.5.2 Contention-less IDM Access

The data DMA, EDAP, AXI slave ports, histograms unit or the core DSP can simultaneously access different blocks without memory conflicts.

## 2.5.3 DMA Queue

The DDMA can accept three pending requests using a dedicated queue.

## 2.5.4 Automatic External Port Selection

The external port used (EDP or AXIm0/1) is selected by matching the external memory address with the address regions configured in the DACU.

## 2.5.5 Configurable Burst Type

Supports INCR or FIXED addressing per DDMA task.

## 2.5.6 Configurable Maximum External Master Port Burst Size

Maximum burst size of up to 256 transfers is supported.

## 2.5.7 Byte-aligned

Supports byte-aligned addressing and a byte-resolution transfer size.

## 2.5.8 Configurable Outstanding Bursts

Maximum External Master Port Outstanding Bursts:

- Supports up to 16 outstanding read bursts.
- Supports four, eight, 16 and 32 outstanding write bursts.

## 2.5.9 Transfer Access Types

- Message: The destination memory is written in sequential order with a 32bit element configured in the DDMA task descriptor. The number of double words to be written is specified by DDTC
- One dimension: The memory is accessed in sequential order
- Two-dimensional: The memory is accessed as an arbitrary two-dimensional tile.
- Data unpacking: The DDMA can write selected bytes read from the source memory to the destination memory. In unpacking operations each line of a two-dimensional tile comprises an integer number of blocks with each block comprising. The UNPACK field in MSS\_2DCFG1 specifies which element from a data block is written to the destination memory. This mode is useful

for extracting a single channel from packed data structure (RGB / RGBA / YCbYCr / Bayer patterns and others).

- Data clipping (padding) : The clipping parameters in the DDMA descriptor define the bytes of a two dimensional tile transfer read from the source memory that are not written to the destination memory. The destination memory is either not written (write-masked) or written with the value PADVAL configured in MSS\_2DCFG4 (padding). Redundant access to source memory still performed since clipping is applied while writing to the destination memory. Together with byte-aligned access clipping and padding allows pixel accurate patches to be merged to IDM or external memory.
- One-bank Tile: IDM data is accessed as a two-dimensional tile with a 32bit width. The tile is allocated to a single IDM bank. Clipping and unpacking cannot be applied.  
In this mode, a single data patch (2D) can be loaded into single IDM data banks, allowing to process 16 separate data patches in parallel.
- Two-bank Tile: IDM data is accessed as a two-dimensional tile with a 64-bit width. The tile is allocated to two adjacent IDM banks. Clipping and unpacking cannot be applied. For more details see [Multi-Scalar optimization technique](#)
- This mode is very similar to One-Bank mode. Each data patch (2D) can be loaded into coupled IDM data banks (two consecutive banks that form a single logical memory bank). Enable to process 8 separate data patches in parallel and to access each of them with a **none-aligned** access. For more details see [Multi-Scalar optimization technique](#)
- Duplicate One-bank Tile: IDM destination data is duplicated to 16 adjacent tiles. Each tile is one IDM bank wide. Clipping and unpacking cannot be applied
- Duplicate Two-bank Tile: IDM destination data is duplicated to 8 adjacent tiles. Each tile is two IDM banks wide. Clipping and unpacking cannot be applied.

### 3. SIMD Programming Overview

Single Instruction Multiple Data (SIMD) is a programmable core architecture that executes the same operation on a vector of inputs. Such architecture is very suitable for computer vision, image processing and CNN applications.

SIMD programming differs in many ways from conventional algorithms programming since efficient implementation requires the algorithm to run on multiple data-paths simultaneously, that required the algorithm's data flow and dependencies to be trickled to exploit the full processing power of the system.

For example, a classic image processing 2D-linear-filter algorithm can be easily partitioned into several independent data-flows, where each flow generates a different result, which is data-independent with the previous one. In the calculation below, each result element  $Z[0]$  depends only on the same elements  $a[0]$ ,  $b[0]$  and  $e[0]$ .

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| a | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|   | * | * | * | * | * | * | * | * |
| b | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|   | + | + | + | + | + | + | + | + |
| e | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|   | = | = | = | = | = | = | = | = |
| Z | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |

Another kind of algorithms are recursive algorithms, meaning that operations frequently depend on previous results. Such algorithms often need to be completely redesigned in order support SIMD systems.

Sorting algorithms are usually recursive and less suitable for SIMD implementation. To overcome that problem, Bitonic-Sort algorithm has been developed, it exploits all of the independency of the data in the sorting process, thus makes it very suitable for parallel computing such as SIMD. See CEVA-CV library for an efficient implementation.

Though the nature of SIMD systems is to process independent data in parallel the XM6 architecture supports several vector operations which are called intra-operations and able to execute operations within a vector, such as `vintramax`, `vintramin` and `vintrasum`

XM6 vectors and VPU instructions are designed in such manner that they support variable degree of parallelism. The supported physical vector is a

256-bit that can be accessed as 32x8bit/16x16bit/8x32bit (fixed or float)/4x64bit. Combination of two vectors can be used to create virtual-vector of 512-bit. This flexibility of core architecture makes it flexible to adapt to large variety of algorithms with different degrees of accuracy and parallelism.

In many cases, SIMD algorithms accelerate the main processing task with such success that the scalar part of the function (prologue and epilogue) become the bottleneck in terms of performance. For exactly such cases, XM6 offers very powerful scalar units that can perform SIMD operations on scalar registers in both fixed and floating-point. Another powerful feature that is used to shorten functions epilogues is vector-store operations that perform casting and saturation before storing the results.

## 4. CEVA-XM6 DSP Vec-C

### 4.1 Vec-C Introduction

Achieving high performance software, especially in the field of DSP and computer-vision is a complex task and the outcome is sometimes code that is very difficult to understand and maintain. Traditionally optimized kernels and sometimes even system layers were written in plain “c” with complex intrinsics or even assembly. Vec-C language comes to improve exactly these issues and to enable complex and maintainable code to be written in C++ (or even vector-C extension) without sacrificing the performance.

Vec-C is based on data-types, instructions and operators all supported by CEVA-VX compilers and simulated on PC platforms using libraries more details in section

The Vec-C language unleashes the power of the CEVA-XM family DSPs while enabling the code to be portable, readable and maintainable for long life cycles.

Writing Vec-C code is simple, it looks like plain C++ code. Some architecture specific instructions, data-types and operators were added as language extensions, combined with optimizing compiler very high performance can be reached.

Simple Vec-C function should look like this:

*Code 4-1: code example to summerize array of integers*

```
int sum_of_array(int *p_array, /* 256bit aligned*/
                int length) /* must be multiple of 8*/
{
    int8 *p8_array = (int8*)p_array;
    int8 sum_int8;
    sum_int8 = 0;
    for (int i = 0; i < length; i += 8)
        sum_int8 += *p8_array++;
    return ((int)vintrasum(sum8, sum_int8));
}
```

### 4.2 Data Types

Native vector of CEVA-XM6 core is of 256-bit, thus the most efficient code should be based on such vectors. Some instructions support 2-vector operations, thus 512-bit vectors are supported as well, due to instructions limitations, not all operations are natively support 512-bit vectors with optimal performance. For software compatibility, 128-bit vectors are also

supported but using these types does not exploit the whole processing power of CEVA-XM6.

**Figure 3: Vector registers partitions**

|        |         |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |         |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 1-bit  | 256-bit |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |         |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|        |         |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    | 128-bit |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| 8-bit  | 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15      | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 16-bit | 15      |    | 14 |    | 13 |    | 12 |    | 11 |    | 10 |    | 9  |    | 8  |    | 7       |    | 6  |    | 5  |    | 4 |   | 3 |   | 2 |   | 1 |   | 0 |   |
| 32-bit | 7       |    |    |    | 6  |    |    |    | 5  |    |    |    | 4  |    |    |    | 3       |    |    |    | 2  |    |   |   | 1 |   |   |   | 0 |   |   |   |
| 64-bit | 3       |    |    |    |    |    | 2  |    |    |    |    |    | 1  |    |    |    |         |    | 0  |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 2: Vec-C data types**

| CEVA Vec-C types                | GCC vector types          | Description                                           |
|---------------------------------|---------------------------|-------------------------------------------------------|
| <b>256 bit – native vectors</b> |                           |                                                       |
| <code>char32</code>             | <code>int8x32_t</code>    | 32 elements of 8-bit signed two's complement integer  |
| <code>uchar32</code>            | <code>uint8x32_t</code>   | 32 elements of 8-bit unsigned integer                 |
| <code>short16</code>            | <code>int16x16_t</code>   | 16 elements of 16-bit signed two's complement integer |
| <code>ushort16</code>           | <code>uint16x16_t</code>  | 16 elements of 16-bit unsigned integer                |
| <code>int8</code>               | <code>int32x8_t</code>    | 8 elements of 32-bit signed two's complement integer  |
| <code>uint8</code>              | <code>uint32x8_t</code>   | 8 elements of 32-bit unsigned integer                 |
| <code>long4</code>              | <code>int64x4_t</code>    | 4 elements of 64-bit signed two's complement integer  |
| <code>ulong4</code>             | <code>uint64x4_t</code>   | 4 elements of 64-bit unsigned integer                 |
| <code>half16</code>             | <code>float16x16_t</code> | 16 element of 16-bit floating point                   |
| <code>float8</code>             | <code>float32x2_t</code>  | 8 elements of float                                   |
| <b>128 bit – half vectors</b>   |                           |                                                       |
| <code>char16</code>             | <code>int8x16_t</code>    | 16 elements of 8-bit signed two's complement integer  |
| <code>uchar16</code>            | <code>uint8x16_t</code>   | 16 elements of 8-bit unsigned integer                 |
| <code>short8</code>             | <code>int16x8_t</code>    | 8 elements of 16-bit signed two's complement integer  |
| <code>ushort8</code>            | <code>uint16x8_t</code>   | 8 elements of 16-bit unsigned integer                 |
| <code>uint4</code>              | <code>uint64x4_t</code>   | 4 elements of 32-bit unsigned integer                 |
| <code>half8</code>              | <code>float16x8_t</code>  | 8 element of 16-bit floating point                    |
| <b>256 bit – double vectors</b> |                           |                                                       |
| <code>short32</code>            | <code>int16x32_t</code>   | 32 elements of 16-bit signed two's complement integer |
| <code>ushort32</code>           | <code>uint16x32_t</code>  | 32 elements of 16-bit unsigned integer                |
| <code>int16</code>              | <code>uint32x16_t</code>  | 16 elements of 32-bit signed two's complement integer |
| <code>uint16</code>             | <code>int32x16_t</code>   | 16 elements of 32-bit unsigned integer                |

| CEVA Vec-C types    | GCC vector types        | Description                                          |
|---------------------|-------------------------|------------------------------------------------------|
| <code>long8</code>  | <code>int64x8_t</code>  | 8 elements of 64-bit signed two's complement integer |
| <code>ulong8</code> | <code>uint64x8_t</code> | 8 elements of 64-bit unsigned integer                |

**Code 4-2: Variable declaration is using a simple C/C++ interface**

```
int8 *p8_array = (int8*)p_array;
int8 sum_int8 = 0;
```

After a vector variable is declared, it can be used as a regular object for assignment, memory access (store/load), operator or function argument.

## 4.3 Packing and Unpacking

CEVA-XM6 core is designed for optimal performance with 256-bit vectors. In many applications it is essential to use longer or shorter vectors, to increase of accuracy (due to higher moment calculation in fixed-point) or to reduce memory usage. For that purpose, the compiler has built-in operators for packing and unpacking vectors. vpack/vunpack operation prototypes (same for `short8`, `short16`, `short32`, `int8`, `int16`):

**Code 4-3: pack / unpack prototypes**

```
char32 vpack(char16 inA, char16 inB);
char16 vunpack_lo(char32 inA);
char16 vunpack_hi(char32 inA);
```

The example following loads takes vectors and concatenates them to create one vector.

```
char16 a [ 97 41 78 19 20 97 8 20 50 75 50 42 49 11 76 74 ]
char16 b [ 61 82 27 53 70 97 19 60 45 34 74 65 37 29 45 15 ]
Char32 z [ 61 82 27 53 70 97 19 60 45 34 74 65 37 29 45 15 97 41 78 19 20 97 8 20 50 75 50 42 49 11 76 74 ]
z = vpack(a,b);
```

Unpacking does the reverse operation

**Code 4-4: unpacking example**

```
a = vunpack_lo(z);
b = vunpack_hi(z);
```

When using the vpack / vunpack the compiler can exploit these operations for better optimization, by using vector allocation instead of vector assignment (vmov) operations

## 4.4 Casting

CEVA-XM compiler does not perform implicit type conversion on vector data types. The reason for the lack of automatic casting is to protect the programmer and to assure that the Vec-C instruction that is used correlates to the DSP instruction-set. The correct choice of data-types and instructions, highly affects the performance.

In Vec-C code the instructions must be called with the correct types, otherwise, the compiler assumes an error and require the programmer to explicitly make the conversion and take into account the possible performance degradation it might cause.

**Code 4-5: vector declaration**

```
short8  my_s8    = 0;
int8    my_i8    = my_s8;           // not allowed
int8    yours_i8 = (int8)my_s8;     // allowed
```

CEVA-XM6 instruction set supports signed (sat) and unsigned (usat) saturation using the vcast() instruction for example:

**Code 4-6: vector casting with saturation**

```
short8 yours_s8 = vcast(sat, my_i8)
```

The following table shows the various conversion supported by vcast()

**Table 3: vcast options**

| Output type | Switch argument | Input type |
|-------------|-----------------|------------|
|             |                 |            |
| ushort16    | in_satu         | char16     |
| uint8       | in_satu         | short8     |
| short16     | in_sat          | int16      |
| short16     | in_sat          | uint16     |
| ushort16    | in_satu         | int16      |
| ushort16    | in_satu         | uint16     |
| char32      | in_sat          | short32    |
| char32      | in_sat          | ushort32   |
| uchar32     | in_satu         | short32    |
| uchar32     | in_satu         | ushort32   |
| short8      | in_sat          | int8       |
| short8      | in_sat          | uint8      |
| ushort8     | in_satu         | int8       |

| Output type | Switch argument | Input type |
|-------------|-----------------|------------|
| ushort8     | in_satu         | uint8      |
| char16      | in_sat          | short16    |
| char16      | in_sat          | ushort16   |
| uchar16     | in_satu         | short16    |
| uchar16     | in_satu         | ushort16   |
| int8        | in_sat          | long8      |
| int8        | in_sat          | ulong8     |
| uint8       | in_satu         | long8      |
| uint8       | in_satu         | ulong8     |
| uint16      | in_satu (2out)  | short16    |
| ushort32    | in_satu (2out)  | char32     |
| ulong8      | in_satu (2out)  | int8       |

## 4.5 Syntax of Instructions

Vec-C instructions prototypes are in a form of c-style function call for example:

```
int16 vmac3(subtract_sw    in_subtract,
            char16         inA,
            unsigned short inB,
            short16        inC,
            unsigned short inD,
            int16           inoutW);
```

Using the following terminology:

`in_subtract` instruction optional switch

`inA ... inF` used for input vectors

`inoutW` is used as input-output vector, equivalent to c++ “by-reference”

The compiler selects the most appropriate assembly instruction, according to the data-type of the arguments used in the code.

“CEVA-XM6™ Architecture Specification Volume II” - chapter 2 explains the syntax and operation of CEVA-XM6 assembly language. It is recommended to read this chapter to get better understanding of the assembly language. Although the Vec-C compiler makes assembly coding obsolete, understanding the assembly language is useful when optimizing and debugging code, especially when compiled with high optimization level (-O4, see compiler flags for more details).

The following is an assembly instruction and the Vec-C equivalent, the colors of the arguments show the related arguments

```
int16 vmac3(subtract_sw, char16, unsigned short, short16, unsigned short, int16);
vmac3 {[subtract]} vAp.[u]c32, rvB.[u]s, vC.[u]s16, rvD.[u]s, vrW0.[u]i8, vrW1.[u]i8 [,?vprX.b16]
```

Table 4: Vec-C and Assembly syntax

| Vec-C                       | Assembly                                            | details                                                                                                                                                                                                                                           |
|-----------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int16</code>          | <code>vrW0.[u]i8,</code><br><code>vrW1.[u]i8</code> | Input and output accumulator consisting of two vector-registers (vrW0, vrW1), the equivalent Vec-C type is <code>int16</code> which is also used as return value                                                                                  |
| <code>subtract_sw</code>    | <code>subtract</code>                               | Optional instruction switch (assembly), in Vec-C signaled as the first argument of the type <code>subtract_sw</code>                                                                                                                              |
| <code>char16</code>         | <code>vAp.[u]c32</code>                             | The first input argument (A), in assembly signaled as “partial” vector (128-bit, 16 elements of char), the Vec-C equivalent is <code>char16</code>                                                                                                |
| <code>unsigned short</code> | <code>rvB.[u]s</code>                               | The second input argument (B) of the type <code>unsigned short</code> the assembly equivalent is unsigned short scalar register                                                                                                                   |
| <code>short16</code>        | <code>vc.[u]s16</code>                              | The third input argument (C), is a 256-bit vector, 16 elements of short                                                                                                                                                                           |
| <code>unsigned short</code> | <code>rvD.[u]s</code>                               | Same type as “B” argument                                                                                                                                                                                                                         |
| <code>vselect()</code>      | <code>[,?vprX.b16]</code>                           | Vector-Predicate, optional. Scalar registers used to mask the required elements in the output that should not be calculated. The Vec-C syntax is compiler directive <code>vselect()</code> more on that in <a href="#">Predicated execution</a> . |

Some Vec-C instructions return several results; therefore, their syntax is slightly different. These instructions do not use the L-value to return the operation result. These instructions use C++ by-reference arguments to return several results, the table below summarizes these instructions, for more details on their operations see the Vec-C Navigator reference guide.

Table 5: Examples for instructions with multiple return-values

| Vec-C prototype examples (each instruction has several variations)                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>void vldchk(const uchar32* inN, uchar32&amp; outZ0, uchar32&amp; outZ1)</code>                                                |
| <code>void vldmulti(addresses_sw in_addresses, short* inM, short16 inN, short8&amp; outZ0, short8&amp; outZ1)</code>                |
| <code>void vldov(const uchar16* inN, ushort16&amp; outZ0, ushort16&amp; outZ1, ushort16&amp; outZ2, ushort16&amp; outZ3)</code>     |
| <code>void vpld(rel_sw in_rel, const unsigned int* inM, short16 inN, uint8&amp; outZ0, uint8&amp; outZ1)</code>                     |
| <code>void vabscmpmov(relation_sw in_relation, char32 inA, char32 inB, char32&amp; outZ0, char32&amp; outZ1)</code>                 |
| <code>void vdeinterleave3(char32 inA, char32 inB, char32 inC, char32&amp; outZ0, char32&amp; outZ1, char32&amp; outZ2)</code>       |
| <code>void vfpextract(float8 inA, uint8&amp; outZ0, int8&amp; outZ1)</code>                                                         |
| <code>void vhfextract(half16 inA, ushort16&amp; outZ0, short16&amp; outZ1)</code>                                                   |
| <code>void vinterleave3(char32 inA, char32 inB, char32 inC, char32&amp; outZ0, char32&amp; outZ1, char32&amp; outZ2)</code>         |
| <code>void vsplit(char32 inA, unsigned char inB, char32&amp; outZ0, char32&amp; outZ1)</code>                                       |
| <code>void vsqrt(ushort16 inA, ushort16&amp; outZ0, short16&amp; outZ1)</code>                                                      |
| <code>void vsqrti(ushort16 inA, unsigned char inB, ushort16&amp; outZ0, short16&amp; outZ1)</code>                                  |
| <code>void vswsub(sat_sw in_sat, char32 inA, char32 inB, char32 inC, unsigned int inE, char32&amp; outZ0, char32&amp; outZ1)</code> |

## 4.6 Operators

The most simple (and sometimes most efficient) way to use the vector-data-types is with the overloaded operators which are built-in the Vec-C compiler and [FIS \(Fast Instruction Simulator\)](#).

The table below lists the overloaded operators. These operators can be used with all vector types, allowing the compiler to choose the most appropriate CEVA-XM opcode in order to execute the operation.

**Table 6: Vec-C overloaded operators**

| Operator           | Description                                                     |
|--------------------|-----------------------------------------------------------------|
| +, +=              | Vector elements SIMD addition (vadd)                            |
| -, -=              | Vector elements SIMD subtraction (vsub)                         |
| *, *=              | Vector elements SIMD multiplication (vmpy, vmac)                |
| >>, >>=<br><<, <<= | Vector elements SIMD shift left or right (vshifl, vshiftr)      |
| ~   ^ &            | Vector elements SIMD logical operations (vor, vand, vxor, vnot) |
| == != > <<br>>= <= | Vector comparison operators                                     |

- Note that implicit type-casting is disabled, thus correct data-types must be used with the operators or appropriate cast should be added. With the exception of casting scalar value to a vector type.

**code 4-7: Vec-C code, using operators**

```

uchar32 *pR, *pG, *pB;
uchar32 *pU, *pV, *pY;
for (i = 0; i < Len; i += 32) {
    uchar32 vR = *pR++;
    uchar32 vG = *pG++;
    uchar32 vB = *pB++;
    uchar32 vY = (uchar32)((((short32)vB * 25 + (short32)vG * 129 + (short32)vR * 66 + (128 + 16 * 256)) >> 8));
    *pU++ = (uchar32)((((short32)vY * (-147) + (short32)vB * 126 + 128 * 256) >> 8));
    *pV++ = (uchar32)((((short32)vY * (-186) + (short32)vR * 160 + 128 * 256) >> 8));
    *pY++ = vY;
}

```

The above snippet is the main loop of RGB to YUV kernel, implemented using operators only

## 4.7 Vector Predicated Execution

Vector Predicator Register (VPR) are a 32-bit/ 16-bit/ 8-bit registers set, used to determine which of the vector elements will be affected by an instruction execution (written back to the output vector). Referenced as [vprX.b32] in the architecture spec. The equivalent of this operator in c is the ternary-operator <condition> ? <if\_true> : <if\_false>

Using VPR is essential in implementations of algorithms that require accurate selection of elements for example:

- Selective filtering
- Data compactness
- Merging and sorting
- Splitting processing of vector to groups (positive/negative, odd/even, below/above thresholds etc.)

## 4.8 Vselect()

The syntax for using a VPR in Vec-C is the **vselect()** statement. The instruction prototype for selecting vector of 8 integers is:

```
int8 vselect(int8 inA, int8 inB, unsigned char inC);
```

The result vector will contain elements from **inA** or from **inB**, according to the value of the corresponding bit in the VPR (**inC**). When the bit is set the element is taken from **inB**, otherwise from **inA**.

for example:

*Code 4-8: vselect example*

```
int8 c, b, odd, msb;
c = 0xC;    // {C, C, C, C, C, C, C, C}
b = 0xB;    // {B, B, B, B, B, B, B, B}
odd = vselect(c, b, 0xAA); // 0b10101010
// {B, C, B, C, B, C, B, C}
msb = vselect(c, b, 0xF0); // 0b11110000
// {B, B, B, B, C, C, C, C}
```

The XM VMOVSELECT instruction is a straight forward implementation of **vselect()**. Nevertheless, it is not the most efficient way to use VPRs since VPRs are supported by all VPU instructions, CEVA-XM optimizing compiler can combine its usage as part of other instructions, whenever possible.

### 4.8.1 Vector comparison

The VPR is used to perform conditional execution in SIMD therefore CEVA-XM6 vector unit support wide range of vector comparison operations. The following is an example of simple **if()** statement, expanded to SIMD.

*Code 4-9 : if statement SIMD extended*

```
// SCALAR
if (abs(a) > 7 || a < 0)
    a = a + 1;

// SIMD
vpr8 = vcmp(lt, a8, 0);           //vpr8= (a8<0)
```

```
vpr8 = vabscmp(gt, vpror, a8, (int8)7, vpr8); //vpr8= (abs(a8)>7) | vpr8
a8 = vselect(a8 + 1, a8, vpr8);
```

More details on vector-comparison instructions at [Assignment and comparing](#)

## 4.8.2 Remarks on vselect() and VPR usage

Using VPRs can be very efficient when used correctly, the following notes can save a lot of effort:

- All **vselect()** operations, require input and output vectors to have the same number of elements and same element size.
- When the results are written to vector-accumulator, (two concatenated vectors) the VPR mask applies to the elements in the accumulator, regardless of the actual vectors it is constructed from.
- Vector-Predicates can be for scalar operations (arithmetic or logic) or as index for memory access (load / store) but these operations may come with a latency cost as shown in the table below.

**Table 7: VPR related latency**

| First operation | Second operation | Latency |
|-----------------|------------------|---------|
| VPU-compare     | SPU-math         | 4       |
| VPU-compare     | LSU-load         | 6       |

There are two ways to overcome the latencies:

1. VPU compare operations offer versatile combinations with logical operands that do not require the usage of scalar-unit (more details at [Assignment and comparing](#))
  2. Use loop unrolling (more details at [Optimizing kernels](#))
- VPRs are special registers in the XM core and are accessible via both scalar units and vector units. That makes VPR usage very flexible and efficient though it sometimes comes with cost of latency
  - When used with intra-operations, the VPR indicates, which of the input vectors elements will be used (as opposed to the output elements with regular operations)
  - Usage with VLSU (vector-load-store-unit)
    - When used with memory store operations, the addresses of the masked operations are accessed, but not written.
    - Vector-Store operation have two predictors, first VPR that determines which elements to store, the second is 1-bit PR, used for conditional execution of the whole instruction. When the PR is masked, none of the addresses will be accessed.
    - Vector-Store post-modification is influenced only by the PR (scalar predicate bit [prB]) and not by the VPR. This is correct for both regular vector-store and parallel-vector-store (see [parallel data access](#) for more details)

- Load operations use the VPR to determine, which element in the written vector is modified. All elements in the addressed memory are accesses, regardless of VPR value.

### 4.8.3 Scalar predication

VPRs can be constant values (see example above), regular scalar registers or the result of vector comparison.

- Scalar predictor. Constructed of 1-bit predictor, referenced as [prX.b] in the architecture spec.  
These predictors are intended to control whether the instruction will perform write-back to the output register (or memory). These predicates are used only by the compiler.
- Scalar SIMD predictor. Constructed of 2-bit predictor, referenced as [prX.b2] in the architecture spec.  
These predictors are used with partitioned registers and are intended to control if the instruction will perform write-back to the output register parts.

## 4.9 Memory Access

With the powerful processing engine that CEVA-XM6 has, efficient data processing can be easily achieved, making memory access the bottleneck of many kernels.

To achieve memory access efficiency the Load-Store-Unit (LSU) must be used as close to its full bandwidth. For that purpose a set of special instructions was developed (see [special data access](#)).

The following list summarizes MSS features:

- 32-bit addressing
- Non-aligned data access (reduced bandwidth)
- Aligned data access (full bandwidth)
- Overlap data load
- Checkered data load
- Parallel data load (16 none-aligned addresses, per unit)
- Parallel data store (16 none-aligned addresses)
- Parallel Coupled-Bank Load (8 non-aligned addresses, per unit)
- Parallel Coupled-Bank Store (8 non-aligned addresses)
- Multi-address Load (2/4/8 addresses , per unit)
- Multi-address Store (2/4/8 addresses)

- Histogram acceleration
- Both scalar and vector predication

**Table 8: CEVA-XM6 memory access bandwidth**

| Unit                | Load bandwidth | Store bandwidth |
|---------------------|----------------|-----------------|
| LS0 – Load only     | 256-bit        | ---             |
| LS1 - Load or Store | 256-bit        | 256-bit         |
| VPU0 – VHIST        | ---            | (2 x) 256-bit   |

More details on memory architecture in [Data memory](#) chapter.

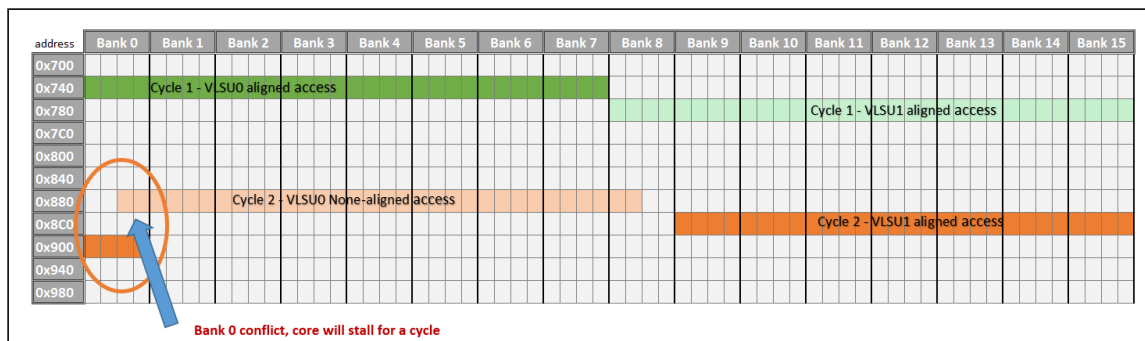
### 4.9.1 Aligned And None-Aligned Vector Data Access

Aligned access is an access to the memory with the same data-type address as the data element being read. In CEVA-XM memory sub system the aligned-access is defined as 32-bit (or 4-byte), as this is the size of the physical data-banks.

Since CEVA-XM architecture supports none-aligned data access, meaning in order to load a vector of 256-bit, the pointer can be only 8-bit aligned for the access to be legal.

That feature comes with a cost, since each data-bank can be accessed by only one unit at the same cycle (VLSU, DMA, VHIST etc..) using none-aligned access requires additional data-bank to be accessed, thus no other unit can access that data-bank at the same cycle. The figure below shows the difference between aligned and none-aligned vector data access.

**Figure 4: none-aligned data access**



There are several ways to avoid such conflicts:

- Avoid none-aligned access by replacing it with aligned access and vector-permutation or manipulating DMA transfers to load the data in such way that it is aligned in the IDM
- Avoid accessing the same data-block at the same cycle, by mapping input and output buffers to different blocks

- Whenever possible, combine scalar access with vector access at the same cycle
- Store operations are written via write-buffer and not at the same cycle as load operations. Therefore sometimes it is more efficient to store and load at the same cycles

## 4.9.2 Parallel Data Access (protected and unprotected)

Parallel access instructions are very powerful tool and enable many scatter-gather operations to be accelerated significantly. The example code below creates a LUT for calculating square-root of an integer and uses it to calculate 8 results using a single instruction, using a base pointer and vector of indexes.

*Code 4-10: example of protected VPLD instruction usage*

```
for (int i = 0; i < 255; i++)
    arr[i] = (int)sqrt(i);

int idx[8] = { 144, 9, 25, 4, 100, 64, 0, 2 };
int8 vidx = *(int8*)idx;

// Parallel protected LUT
*(int8*)sqrt_vpld = vpld((int*)arr, vidx);
// Reference calculation
for (int i = 0; i < 8; i++)
    sqrt_scalar[i] = arr[idx[i]];
```

The parallel access instructions are divided into two groups:

- Protected access  
XM6 Vector Load/Store (VLSU) instructions can be used in a straightforward manner as used in the example above. This method is very simple on one hand but on the other hand might cause banks contentions. The core handles these contentions by initiating stall cycles.
- Unprotected (relative) access  
By using the `rel` switch, the index vector is used to calculate a separate address for each memory bank. In this access method, there is no possible danger of bank-contention, but it requires special memory arrangement and possible data duplication per bank (x16) or per 2 banks (x8).

The example below shows two equivalent instructions with almost the same results (see the explanation below). The first uses regular VPLD (protected access) while the second instruction uses relative-parallel access (unprotected). The difference can be seen in the index vectors. While the regular access calculates the offset from the base address, for relative access

the offset is calculated relative to the beginning of each bank. Please see the offset calculation difference for element #9.

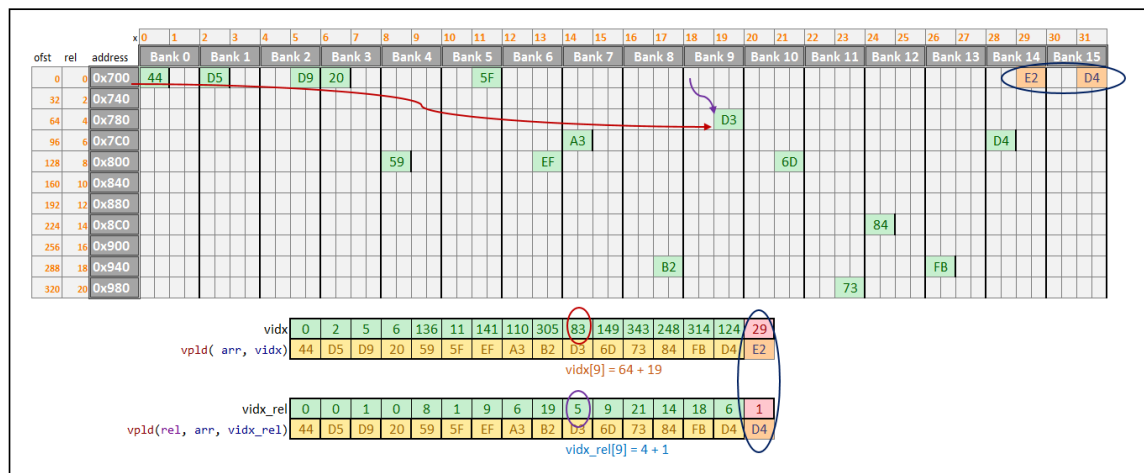
**Table 9: relative and absolute address calculations**

| Base  | Column offset | Row offset | Element size | Physical address                                  |
|-------|---------------|------------|--------------|---------------------------------------------------|
| 0x700 | 19            | 64         | 2            | $0x700 + 2 \cdot (64 + 19) = 0x7A6$               |
| 0x700 | 1             | 4          | 2            | $0x700 + \text{bank\_9}(2 \cdot (4 + 1)) = 0x7A6$ |

The last element in the result vector is different on purpose, to emphasize the difference between the two instructions. The protected access loads from address 0x73A ( $0x700 + 2 \cdot 29$ ) in element 15, which conflicts with element 14 (that loads from bank 14 as well). This conflict will impose a stall cycle, to preserve correctness.

On the other hand, unprotected (relative) accesses 0x73E which is the second element of bank15, thus no conflict can occur.

**Figure 5: parallel and relative-parallel access**



## 4.9.3 Histogram - VHIST

Histogram calculations require atomic-operation of Load-Add-Store in sparse address range, which overloads the memory. To overcome that problem a special mechanism has been added to VPU0. Please note that although this is a memory access operation it uses VPU0 and not the VLSUs, that allows us to design kernels that use both VLSU and VPU0 to access separate IDM memory blocks in the same cycle.

Histograms are arranged in the memory in a distributed structure of 16 histograms, one for each memory bank (or 32 if two memory blocks are used). At the end of histogram calculation all histograms are accumulated

into a single unified histogram (see CEVA-CV kernels and algorithms for working example).

Several important notes on the VHIST mechanism

- VHIST instruction can be used with “init” switch to initialize the memory region of the histogram, without reading it first. This instruction can be used to initiate large memory regions.
- In the instruction variation, that updates 32 bins (and not 16), two separate memory blocks are being accessed at the same cycle.
- VHIST can add to each bin either:
  - 1 / -1
  - Weight / -Weight

#### 4.9.4 Important notes on memory usage

In order to avoid memory contentions, several rules must be followed:

- Avoid placing memory tables (structures) that are accessed simultaneously in the same memory block, to avoid block-contentions between the two LSUs.
- The programmer cannot control which of the LSU will be used, it is up to the compiler to decide. However, the kernel implementation should take under consideration the overall bandwidth and the latencies. Well balanced code makes it easy for the compiler to optimize the assembly packets
- Two important compiler features can assist the programmer to hint the compiler how the memory is used. More details on about `__restrict__` keyword and “`mem_block()`” attribute in [Compiler attributes](#) chapter.
- When using DMA to pre-fetch data from external memories, the work phases should be mapped to different memory-blocks to avoid CORE-DMA contentions
- When designing LUT that need to be accessed via Vector-Parallel-Load (VPLD), it is recommended to duplicate the table into multiple data-banks (or coupled banks) to avoid bank-contentions
- Tables that have small data-types (such as 8-bit) should be considered to widen into 32-bit, to match bank width of 32-bit and avoid accessing the same aligned address (32-bit) with contention. With small tables it is recommended to use `vlut()` instructions whenever possible.
- When duplicating tables (especially LUT), the size might be marginal for typical DSP chip, therefore techniques to compress tables or to address tables using slightly more complicated calculation might lead to smaller tables and better performance

- When duplicating tables and increasing the memory footprint, some data compacting techniques can be applied to keep the overall memory footprint within chip limits

Keeping these rules in mind while designing our software is essential and can lead to meaningful improvement in overall performance

## 5. Instructions Overview

This chapter is brief reference for XM6 instructions grouped according to their designated functionality. Detailed information on each instruction can be found in the “CEVA-XM6 Vec-C instructions reference”.

**Table 10: instructions glossary**

| Notation       | details                                                                                                                                                                                                                                                                                      |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Units notation | I – VPU0<br>II – VPU0, VPU1<br>III – VPU0, VPU1, LVPU<br>L – LSU0<br>LL – LSU0, LSU1                                                                                                                                                                                                         |
| a,b,c,d,e,f    | Notation for instructions input arguments                                                                                                                                                                                                                                                    |
| z,y,u,v        | Notation for instructions output arguments                                                                                                                                                                                                                                                   |
| psl            | Post-Shift-Logical, toggles the post-shift from arithmetic (default) to logical                                                                                                                                                                                                              |
| rnd            | Rounding, initializes the output vector to rounding value before accumulating the instructions result.<br>The rounding value is defined as $1 \ll (\text{shift\_val} - 1)$<br>shift_val by default is the element size of the result vector (in bits) or other argument value when specified |
| accumulate     | Enables accumulation in instructions that do not accumulate by default                                                                                                                                                                                                                       |
| noacc          | Disables the default accumulation to the result vector                                                                                                                                                                                                                                       |
| subtract       | Sets the instruction's operation to be subtracted instead of added (default) to the output vector-accumulator                                                                                                                                                                                |
| init           | Same as rnd                                                                                                                                                                                                                                                                                  |
| splitsrc       | Treats one or more of the input vectors as two vectors, high-part and low-part                                                                                                                                                                                                               |
| square         | Switches calculation of square value of the input arguments (ISA specific)                                                                                                                                                                                                                   |
| address        | vadd specific switch, see ISA for more details                                                                                                                                                                                                                                               |
| sat            | Performs signed saturation on the instruction result $[\text{min}(\text{type})..\text{max}(\text{type})]$                                                                                                                                                                                    |
| satu           | Performs unsigned saturation on the instruction result $[0..\text{max}(\text{type})]$                                                                                                                                                                                                        |
| $z[k]$         | Notation for element #k in the output vector                                                                                                                                                                                                                                                 |
| $z_i[k]$       | Notation for output accumulator, also denoted as inoutW                                                                                                                                                                                                                                      |
| $z_{i-1}[k]$   | Notation for input (i-1 denotes previous value) accumulator, also denoted as inoutW                                                                                                                                                                                                          |
| $a[k]$         | Notation for element #k in the first vector input argument, other input arguments are marked as $b[k], c[k]...$                                                                                                                                                                              |

| Notation           | details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-----------------|----------------|-----------------------------------|------------------|-----------------------|----------|----------------|-------------------|-------------|-----------|---------------------|--------------------|---------------|----------|------------------------------------------------------------|
| $x = \{a, b\}$     | Vector concatenation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| $a[k][n:m]$        | Notation for set of bits, located in vector $a[]$ , element $\#k$ , from bit $\#m$ to bit $\#n$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| $X[k]$             | Capital letter denotes a 2D vector, usually defined as two concatenated vectors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| $g[0], g[1]$       | Denotes element of partitioned scalar register (2x16-bit or 4x8-bit)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| $\text{abs}(x)$    | Absolute value of $x$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| $\text{sat}(x)$    | <p>Saturation values are defined according to the element type of vector <math>Z</math>, maximum and minimum possible values.</p> <table> <tr> <th>Type (bytes)</th><th>Range of Values</th></tr> <tr> <td>signed int (4)</td><td>[−2,147,483,648 .. 2,147,483,647]</td></tr> <tr> <td>unsigned int (4)</td><td>[0 ... 4,294,967,295]</td></tr> <tr> <td>char (1)</td><td>[−128 ... 127]</td></tr> <tr> <td>unsigned char (1)</td><td>[0 ... 255]</td></tr> <tr> <td>short (2)</td><td>[−32,768 .. 32,767]</td></tr> <tr> <td>unsigned short (2)</td><td>[0 .. 65,535]</td></tr> <tr> <td>long (8)</td><td>[−9,223,372,036,854,775,808 ... 9,223,372,036,854,775,807]</td></tr> </table> | Type (bytes) | Range of Values | signed int (4) | [−2,147,483,648 .. 2,147,483,647] | unsigned int (4) | [0 ... 4,294,967,295] | char (1) | [−128 ... 127] | unsigned char (1) | [0 ... 255] | short (2) | [−32,768 .. 32,767] | unsigned short (2) | [0 .. 65,535] | long (8) | [−9,223,372,036,854,775,808 ... 9,223,372,036,854,775,807] |
| Type (bytes)       | Range of Values                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| signed int (4)     | [−2,147,483,648 .. 2,147,483,647]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| unsigned int (4)   | [0 ... 4,294,967,295]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| char (1)           | [−128 ... 127]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| unsigned char (1)  | [0 ... 255]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| short (2)          | [−32,768 .. 32,767]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| unsigned short (2) | [0 .. 65,535]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |
| long (8)           | [−9,223,372,036,854,775,808 ... 9,223,372,036,854,775,807]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |              |                 |                |                                   |                  |                       |          |                |                   |             |           |                     |                    |               |          |                                                            |

## 5.1 Assignment and Comparing

vabscmp, vabscmpmov, vabssubcmp, vacccast, vaccmov, vcast, vcmp, vcmpacc, vcmpmov, vmov, vmovselect, vclip

## 5.2 Permutation

Data permutation and reordering can make is essential to achieve high efficiency implementation that exploits all the processing power of the DSP.

### 5.2.1 Vector Permutation

There are two powerful vector permutation instructions

| Instruction | Units | Switches | Operation                          | Details   |
|-------------|-------|----------|------------------------------------|-----------|
| vperm       | III   |          | $x = \{a, b\}$<br>$z[k] = x[c[k]]$ | $c[k]$ is |

| Instruction    | Units | Switches             | Operation                                                                                                                   | Details                                                                                                                              |
|----------------|-------|----------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>vlut</b>    | III   |                      | $x = \{a, b\}$<br>$z[k] = x[c[k] - d]$                                                                                      | Vector c[] is index, d is a scalar value that sets offset for c[]<br>Can be used to perform LUT operation, without accessing memory. |
| <b>vlutmac</b> | II    | nomp<br>cross<br>psl | $x = \{a, b\}$<br>$z_i[k] = z_{i-1}[k] + c[k]x[c[k]]$<br>$\quad + d[k]x[d[k]]$<br>Or<br>$z_i[k] = z_{i-1}[k] + d[k]x[c[k]]$ | Used in bilateral filter                                                                                                             |

## 5.2.2 Interleaving

Figure 6: interleaving instructions

| Instruction           | Units | Switches    | Operation                                                                       | Details                                                                                                                                  |
|-----------------------|-------|-------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vinterleave</b>    | III   | satu<br>sat | $z[2k] = a[k]$<br>$z[2k + 1] = b[k]$                                            | The number of elements in a[] and b[] is half of z[]                                                                                     |
| <b>vdeinterleave</b>  | III   | odd         | $z[k] = a[2k]$<br>or<br>$z[k] = a[2k + 1]$                                      |                                                                                                                                          |
| <b>vdeinterleave3</b> | II    |             | $x = \{a, b, c\}$<br>$z[k] = x[3k]$<br>$y[k] = x[3k + 1]$<br>$u[k] = x[3k + 2]$ | Deinterleaves 3 input vectors (a,b,c) into 3 output vectors (z,y,u). Designed for interleave to planar format conversion. (rgb) -> R,G,B |
| <b>vinterleave3</b>   | II    |             | $x = \{z, y, u\}$<br>$x[3k] = a[k]$<br>$x[3k + 1] = b[k]$<br>$x[3k + 2] = c[k]$ | Performs interleaving of 3 vectors. Planar to interleaved format conversion. R,G,B -> (r,g,b)                                            |

## 5.3 Bit Manipulation

Table 11: bit manipulation instructions

| Instruction     | Units | switches | Operation                              | Details               |
|-----------------|-------|----------|----------------------------------------|-----------------------|
| <b>vand</b>     | III   |          | $z[k] = a[k] \& b[k]$                  |                       |
| <b>vcntbits</b> | III   |          | $z[k] = \text{count\_set\_bits}(a[k])$ | Also scalar variation |
| <b>vextract</b> | III   |          | $z[k] = a[k][c:b]$                     |                       |

| Instruction | Units | switches              | Operation                                                                                                                                                                                                                                                        | Details                                                                                                                                                  |
|-------------|-------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| vffb        | III   | sb<br>clr_set<br>sign | $z[k] = \text{first\_set\_bit\_from\_msb}(a[k])$<br>$z[k] = \text{first\_set\_bit\_from\_lsb}(a[k])$<br>$z[k] = \text{first\_clear\_bit\_from\_msb}(a[k])$<br>$z[k] = \text{first\_clear\_bit\_from\_lsb}(a[k])$<br>$z[k] = \text{first\_none\_sign\_bit}(a[k])$ | Locates the position of the first set/clear bit , from msb/lsb.<br>Or first non-sign bit<br>Scalar version exists too.                                   |
| vhamm       | II    | accumulate            | $z_i[k] = z_{i-1}[k] + \text{count\_bits}(a[k]$<br>$\oplus b[k])$                                                                                                                                                                                                | Hamming distance calculation, per 32-bit elements                                                                                                        |
| vsplit      | II    |                       | $z[k] = a[k][:b + 1]$<br>$y[k] = a[k][b:0]$                                                                                                                                                                                                                      |                                                                                                                                                          |
| vxor        | III   |                       | $z[k] = a[k] \oplus b[k]$                                                                                                                                                                                                                                        |                                                                                                                                                          |
| vinser      | III   |                       | $z_i[k][c + b:b] = a[k][c:0]$                                                                                                                                                                                                                                    |                                                                                                                                                          |
| vnand       | III   |                       | $z[k] = \sim(a[k] \& b[k])$                                                                                                                                                                                                                                      |                                                                                                                                                          |
| vnor        | III   |                       | $z[k] = \sim(a[k]   b[k])$                                                                                                                                                                                                                                       |                                                                                                                                                          |
| vnot        | II    |                       | $z[k] = \sim a[k]$                                                                                                                                                                                                                                               |                                                                                                                                                          |
| vor         | III   |                       | $z[k] = a[k]   b[k]$                                                                                                                                                                                                                                             |                                                                                                                                                          |
| vaccshiftl  | II    |                       | $z[k] = \begin{cases} a[k] \ll b[k], & 0 < b[k] < m \\ a[k] \gg  b[k] , & -m < b[k] < 0 \\ 0, & m < b[k] \\ -1, & m > b[k] \end{cases}$                                                                                                                          | Safe vector element shift. For negative shift value, will shift right. Perform clipping when shift out of range. Works on accumulators (512-bit vectors) |
| vaccshiftr  | II    |                       | $z[k] = \begin{cases} a[k] \gg b[k], & 0 < b[k] < m \\ 0, & m < b[k] \\ -1, & m > b[k] \end{cases}$                                                                                                                                                              | Safe vector element shift. For negative shift value, will shift right. Perform clipping when shift out of range. Works on accumulators (512-bit vectors) |
| vshiftl     | III   |                       | $z[k] = \begin{cases} a[k] \ll b[k], & 0 < b[k] < m \\ a[k] \gg  b[k] , & -m < b[k] < 0 \\ 0, & m < b[k] \\ -1, & m > b[k] \end{cases}$                                                                                                                          | Safe vector element shift. For negative shift value, will shift right. Perform clipping when shift out of range.                                         |
| vshiftr     | III   |                       | $z[k] = \begin{cases} a[k] \gg b[k], & 0 < b[k] < m \\ 0, & m < b[k] \\ -1, & m > b[k] \end{cases}$                                                                                                                                                              | Safe vector element shift. For negative shift value, will shift right. Perform clipping when shift out of range.                                         |

## 5.4 Arithmetic

The CEVA-XM6 is a versatile and powerful fixed point math with a lot of instructions that combine various arithmetic and multiplication operations. The table below summarizes the various combinations for convenient instruction search.

Table 12: Arithmetic instructions index

| Instruction | Units | switches                                 | Operation                                                                                  | Details                                                                                                                                  |
|-------------|-------|------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| vabssub     | III   |                                          | $z[k] = abs(a[k] - b[k])$                                                                  |                                                                                                                                          |
| vabssubacc  | II    |                                          | $z_i[k] = z_{i-1}[k] + abs(a[k] - b[k])$                                                   |                                                                                                                                          |
| vaccadd     | II    |                                          | $z[k] = a[k] + b[k]$                                                                       | Vector addition of different types (same number of elements)                                                                             |
| vaccsub     | II    |                                          | $z[k] = a[k] - b[k]$                                                                       | Vector accumulators (512-bit) subtraction                                                                                                |
| vadd        | III   | x2<br>x4<br>x8                           | $z[k] = a[k] + b[k]$                                                                       | Vector addition of same types. x2,x4,x8 switches are needed to generate multi-load addresses                                             |
| vaddmac     | II    | ps1<br>rnd                               | $z_i[k] = z_{i-1}[k] + c[k](a[k] + b[k])$                                                  |                                                                                                                                          |
| vaddmpy     | II    | ps1<br>rnd                               | $z[k] = c[k] * (a[k] + b[k])$                                                              |                                                                                                                                          |
| vaddsat     | III   |                                          | $z[k] = sat(a[k] + b[k])$                                                                  | Saturation values derived from z element type                                                                                            |
| vcomplexmpy | II    | accumulate<br>re<br>im<br>cmplx<br>ps1   | $z[2k] = a[2k]b[2k] - a[2k + 1]b[2k + 1]$<br>$z[2k + 1] = a[2k]b[2k + 1] + a[2k + 1]b[2k]$ | Complex multiplication. Input vectors are either interleaved or planar                                                                   |
| vmac        | II    | subtract<br>ps1<br>rnd                   | $[k] = z_{i-i}[k] \pm a[k]b[k]$                                                            | Multiply and accumulate<br>Optional post shift switch                                                                                    |
| vmac3       | II    | splitsrc<br>square<br>subtract<br>ps1    | $z_i[k] = z_{i-i}[k] \pm (a[k]b[k] + c[k]d[k])$                                            | Multiply two pairs of sources and accumulate. Sources can be of various types                                                            |
| vmac5       | II    | noacc<br>ps1                             | $z_i[k] = z_{i-i}[k] + a[k]e[0] + b[k]e[1] + c[k]f[0] + d[k]f[1]$                          | Multiply four pairs of sources and accumulate (optional). Sources e and f are partitioned registers of 32-bit (each partition is 16-bit) |
| vmad        | II    | init<br>square<br>splitsrc<br>ps1<br>rnd | $z[k] = a[k]b[k] + c[k]d[k]$                                                               | Multiply two pairs of sources. Sources can be of various types<br>Usually used for initializing VMAC loops.                              |
| vmpy        | II    | sb<br>splitsrc<br>ps1<br>rnd             | $z[k] = a[k]b[k]$                                                                          | sb – returns the MSB part of 64-bit value                                                                                                |

| Instruction | Units | switches                        | Operation                                                     | Details                                                                                                                                                      |
|-------------|-------|---------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| vmpyadd     | II    | psl                             | $z[k] = a[k]b[k] + c[k]$                                      | Argument c is either vector or scalar                                                                                                                        |
| vmpymat4x4  | II    | accumulate<br>part<br>transpose | $Z_{4x4} = A_{4x4}B_{4x4}$                                    | Performs 4x4 matrix multiplication.<br>Can transpose matrix A.<br>when the result is written to long (64-bit) data type, requires 2 instructions to complete |
| vmpynorm    | II    | psl<br>rnd                      | $z[k] = (a[k]b[k]) \gg c[k]$                                  |                                                                                                                                                              |
| vneg        | III   |                                 | $z[k] = -a[k]$                                                | Negation of signed vector                                                                                                                                    |
| vsequence   | II    |                                 | $z[k] = a * k$                                                | Generate a vector values                                                                                                                                     |
| vsqacc      | II    | splitsrc<br>subtract<br>psl     | $z_i[k] = z_{i-i}[k] \pm (a[k]^2 + b[k]^2 + c[k]^2 + d[k]^2)$ | Accumulate four square vectors                                                                                                                               |
| vsub        | III   |                                 | $z[k] = a[k] - b[k]$                                          | Vector subtraction                                                                                                                                           |
| vsubmac     | II    | psl<br>rnd                      | $z_i[k] = z_{i-i}[k] - (a[k] - b[k])c[k]$                     |                                                                                                                                                              |
| vsubmpy     | II    | psl<br>rnd                      | $z[k] = (a[k] - b[k])c[k]$                                    |                                                                                                                                                              |
| vsubsat     | II    | psl                             | $z[k] = \text{sat}(a[k] - b[k])$                              |                                                                                                                                                              |
| vsubsquare  | II    | accumulate<br>subtract<br>psl   | $z_i[k] = z_{i-i}[k] \pm ((a[k] - b[k])^2 + (c[k] - d[k])^2)$ |                                                                                                                                                              |

Table 13: glossary

## 5.5 Intra Operations

SIMD operations are usually element-wise where each element in each result vector is calculated using the same element index in the other arguments.

Intra-operations are different in the sense that the result is not calculated element wise but vector wise.

VINTRAMIN / VINTRAMAX – in these instructions, there is only one input vector-argument and one output argument, either scalar or vector filled by scalar.

VINTRASUM – calculates the sum of all the elements inside the vector. This instruction return a scalar value only when summing vector of integers (int8).

When other vector types are being summed, the output is written into a vector of integers (int8), this is due to long latency caused when summing many elements. Therefore in order to summarize vector of more than 8 elements (short16, short32, char32, 2\*char32), it is necessary to calculate intermediate result of int8 and then summarize it to a scalar.

**Code 5-1:vintrasum() example**

```
short minmax_val[16] = { 0, -1, 77, 88, 88, -2, -99, 12, 3, 1, -7, 89, 994, -12, 9, 1 };
short16 val = *(short16*)minmax_val;
int8 temp_val = vintrasum(val);           // {-1, 165, 86, -87, 4, 82, 982, 10}
int sum_val = vintrasum(temp_val);        // 1241
```

**Table 14: intra instructions**

| Instruction | Units | switches | Operation                                                                                                    | Details                                                                       |
|-------------|-------|----------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| vintramax   | II    |          | $z[0] = \max(a[0], a[1], \dots a[N])$                                                                        | When output is of vector type the maximal value is duplicated to all elements |
| vintramin   | II    |          | $z[0] = \min(a[0], a[1], \dots a[N])$                                                                        | When output is of vector type the minimum value is duplicated to all elements |
| vintrasum   | II    |          | $z[k] = z[0] = \text{sum}(a[0], a[1], \dots a[8])$<br>$z[k] = \text{sum}(a[4k], a[4k + 1], \dots a[4k + 3])$ | Refer to the ISA for detailed description                                     |

## 5.6 Convolution sliding-window

2D data convolution is in the heart of image processing and CNN algorithms, the sliding-window-generic mechanism was developed in order to achieve maximal efficiency in terms of power, bandwidth and cycles.

Each VPU of the CEVA-XM6 core is equipped with powerful convolution engine capable of calculating up to 32 results, each is accumulation of 4-tap filter operation.

In this chapter, we'll focus on the sliding-window-generic instruction (vswgenmac) which is designed to cope with two major problems:

- **Data Reuse** – One of the limitations on filters is the large amount of data that need to be read and written. In many cases, such as CNN filters, the input data patches are very small and each patch need to be filtered several times using different coefficients, sometimes with decimation and clipping.
- **Multipliers Utilization** – the intensity of filtering requires multipliers to be utilized to their maximum capacity in order to achieve power efficiently.

These problems are solved using flexible configuration of the convolution mechanism and flexible data memory access via VLDMULTI and VPLD instructions.

In order to configure the sliding-window, it is necessary to understand the following terms:

### 5.6.1 Convolution of an element

The basic convolution process of each result element can be generalized to the formula:

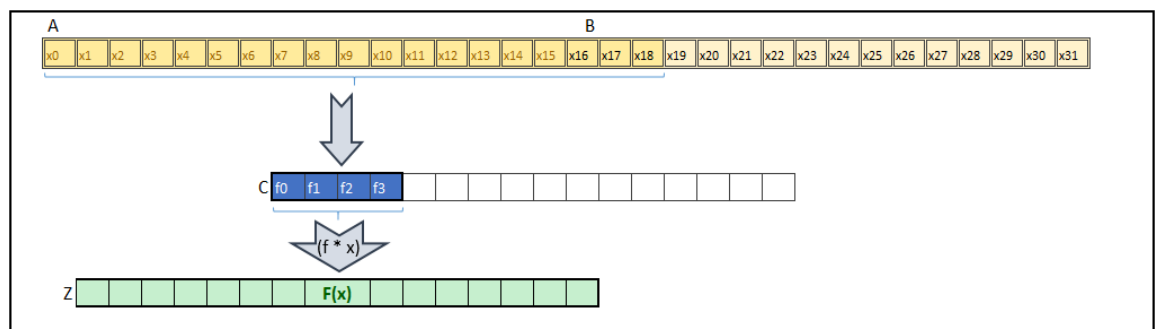
$$z_i[k] = z_{i-1}[k] + \sum_{n=0}^3 f[q+n] * X[r + (1+s)k + P_n]$$

|         |                                                                        |
|---------|------------------------------------------------------------------------|
| $z_i[]$ | Result accumulator (inoutW) of iteration i                             |
| $X[r]$  | Input slice [inA, inB], starting at position r. in 2D mode X is matrix |
| $f[q]$  | filter taps [inC , inD] , starting at position q                       |
| $P_n$   | 2D position defined by the pattern (see below)                         |
| $s$     | Number of elements to skip                                             |
| $k$     | Index of the element position in the output vector                     |

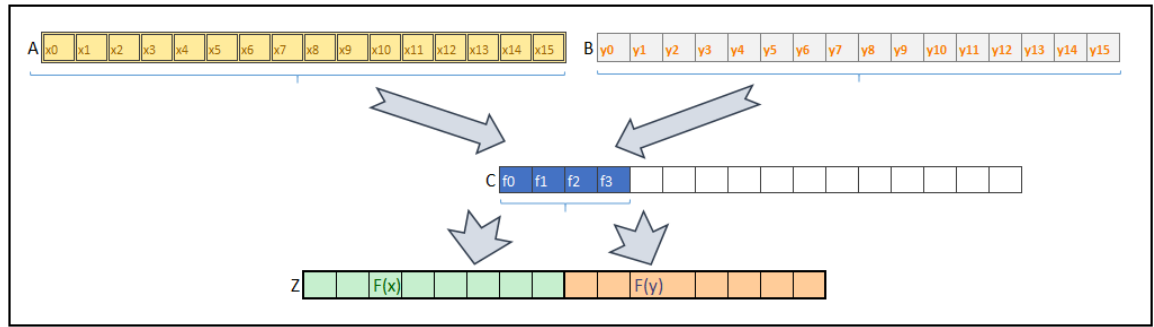
### 5.6.2 Data Slices

The input vectors can be divided into logical independent slices (1,2,4,8) of data, each is processed using same configuration and possibly different coefficients.

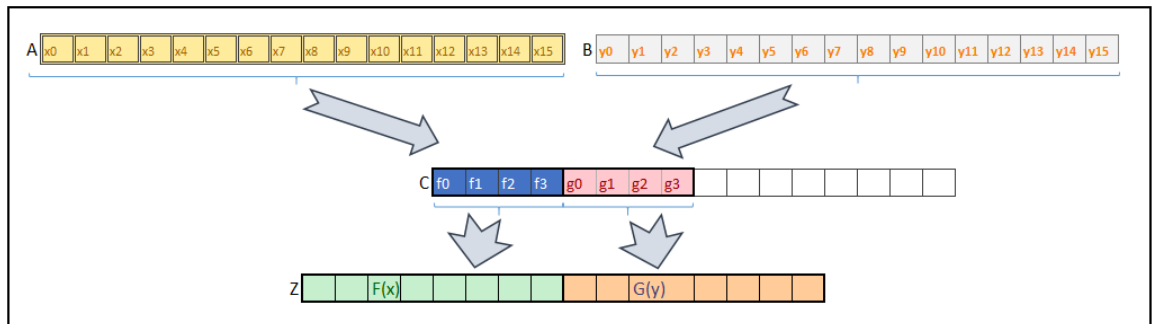
**Figure 7: single slice 1D, vector-B is the continuation of vector-A**



**Figure 8: two slices with single filter (reuse)**



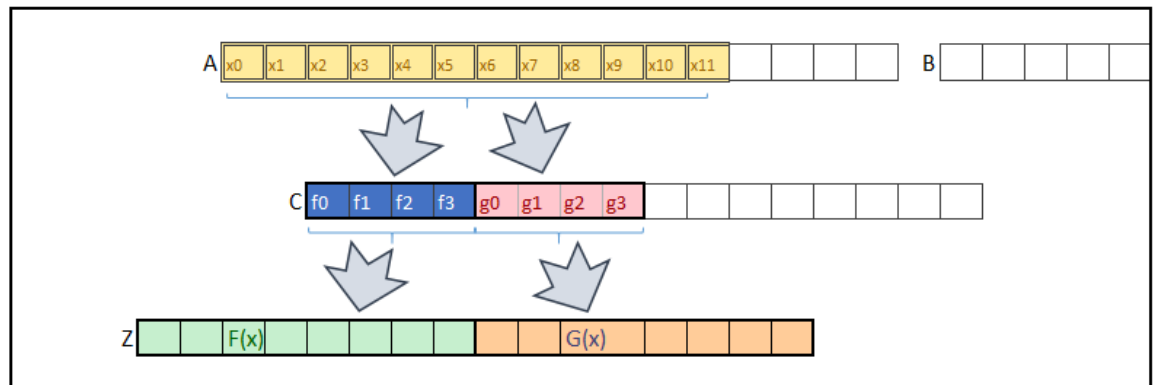
**Figure 9: two slices, two filters, no filter reuse**



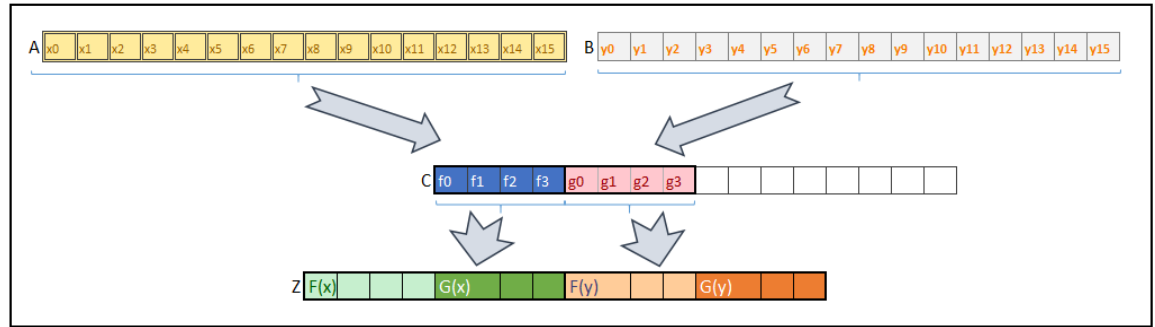
### 5.6.3 Multi Filter

Filter the same input slice using several coefficient sets, allowing each slice to be filtered several times with different coefficients.

**Figure 10: Multi-filters, single slice**



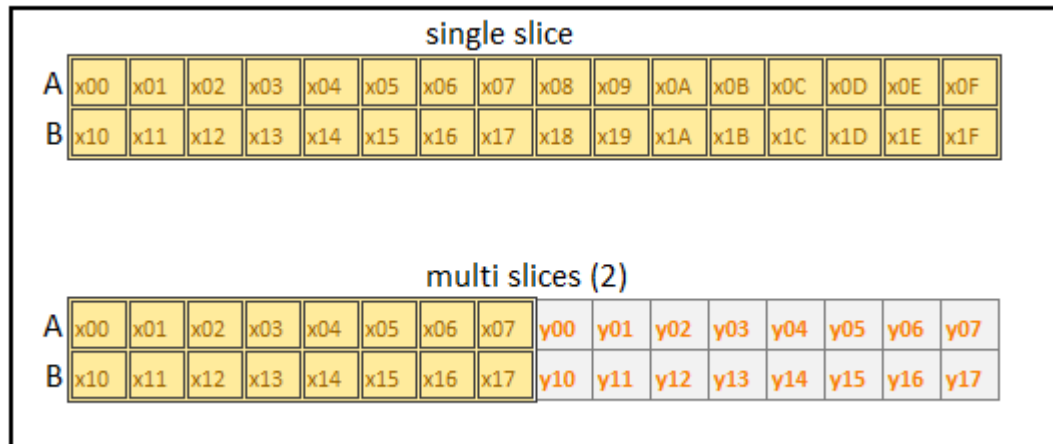
**Figure 11: Multi-filters, multi-slices**



## 5.6.4 Filter dimensions

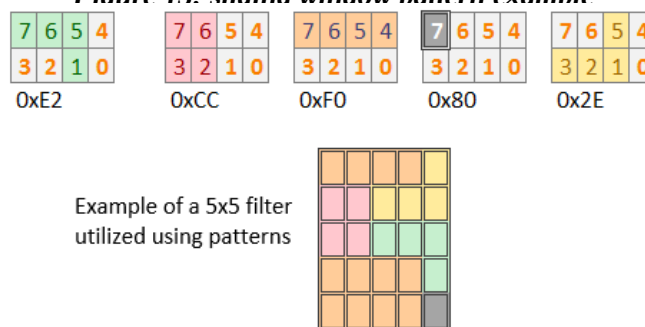
Number of input data dimension – 1D / 2D. When the dimension is set to 2D the first input (inA) is set as top row and the second input (inB) is the bottom row.

Figure 12: 2D filters



## 5.6.5 Filter Pattern

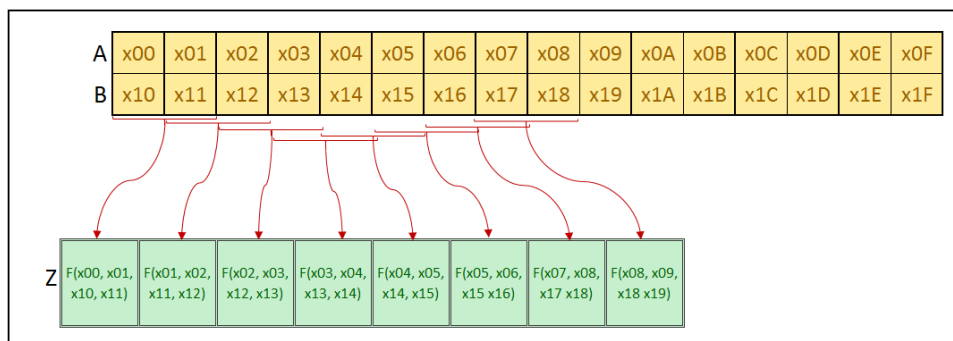
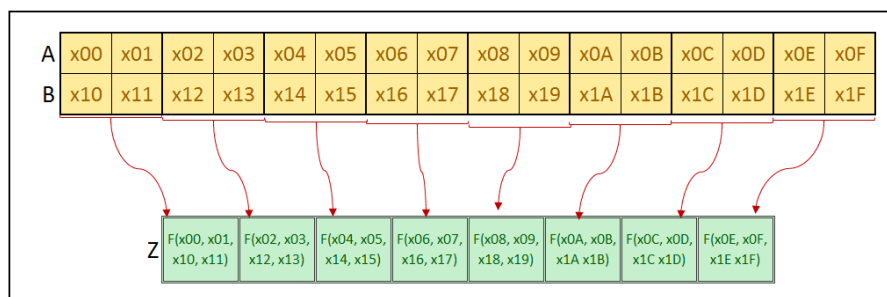
Spatial filter taps topology, each one of the 4 filter coefficients receives its spatial location, relative to filter output location from the pattern field. See field “PATTERN” in configuration word “F” in the table below. The following example shows how a 2D 5x5 filter can be utilized with 89% using pattern mechanism

**Figure 13: sliding window pattern example**

The filter above is performed in 7 cycles, in 6 of them all 4 multipliers are utilized and in the last only one tap is used (25/28)

## 5.6.6 Steps

Input steps can be configured for filtered decimation, used in many CNN layers. For example:

**Figure 14: 2D filter, pattern=0xCC, skip=0****Figure 15: 2D filter, pattern=0xCC, skip=1**

## 5.6.7 Clipping

MaxZ sets the option of clipping negative results to zero, as Used by many CNN layers.

$$Z = \text{Max}(\text{result}, 0)$$

## 5.6.8 VSWGGENMAC control words

The instruction has two configuration registers called “E” and “F” and three switches, one for each filter life cycle:

- INIT – instruction switch, used at the first filter iteration, initialize the accumulator vector iteration, according to the supplied value (a) in the E register.
- ACCUMULATE – used in each iteration of the filter except the first and the last. The convolution results are accumulated to the destination vector
- Normalization (PSL for logical shift) – instruction switch, usually used in the last filter iteration to perform post-shift and reduce the result vector element size (for example from 32bit to 16bit). The shift value is set in the first control register (E). If logical shift is required, the PSL switch is set, otherwise the default is arithmetic shift.

Control register 0 (aka rE) defines the general filter parameters:

**Table 5-15: Vswgenmac rE Register Register Field Description**

| Name                  | Bits/Location | Description                                                                                                                                      |
|-----------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| INIT / PSL VALUE (a)  | 0..7          | when “init” switch is used it defines the initial value for the accumulator ( $2^a$ )<br>when “psl” switch is used, defines the post-shift value |
| DATA OFFSET (d)       | 8..15         | Starting offset in the data input vector                                                                                                         |
| COEFF OFFSET (c)      | 16..23        | Starting offset in the coefficients input vector                                                                                                 |
| FILTER DIMENTION (m)  | 24            | 2D filter (m=0)<br>1D filter (m=1)                                                                                                               |
| COEFFICIENT REUSE (r) | 25            | Use same coefficient for all slices                                                                                                              |
| MAXZ (z)              | 26            | Round negative results to zero                                                                                                                   |

Control register 1 (aka rF) defines the slices, pattern and coefficient sets:

**Table 5-16: VSWGGENMAC rF Register Field Description**

| Name             | Bits/Location | Description                                   |
|------------------|---------------|-----------------------------------------------|
| COEFFICIENT SETS | 0..7          | Number of coefficient sets 1,2,4,8,16 ...     |
| STEP             | 8..15         | Number of elements to skip in the input slice |
| NUMBER OF SLICES | 16..23        | Number of slices (1,2,4,8,16)                 |

| Name           | Bits/Location | Description                                                                                                                              |
|----------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <i>PATTERN</i> | 24..31        | 2 rows of 4 bits, representing spatial positions of the filter coefficients.<br>Only 4 bits can be set one for each coefficient position |

## 5.6.9 Important notes

- Border conditions -  
The flexibility in configuring the input vector positions, steps and pattern may cause the input or coefficient coordinates to overflow outside the vector sizes.  
When coordinates overflow occurs, the slices data is being read in wrap-around manner. Input vectors [A,B] the wrap-around is from B to A, in coefficients vector [C,D] the wrap around is from C to D.
- 8-bit filters -  
when both input slices (A,B) and coefficients (C,D) are of 8-bit elements (char32 or uchar32), then the operation will be of 2-taps filter and with 32 output results.
- Coefficients offset -  
There is a restriction on the coefficients offset parameter that requires it to be a multiplication of the actual number of coefficients in the filter (4-tap or 2-tap)

## 5.7 Other Sliding-Window instructions

All Sliding-Window instructions are based on the VSWGGENMAC configuration, with several slight differences.

### 5.7.1 Sum of difference sliding window

The basic SWGENSAD process of each result element can be generalized to the formula:

$$z_i[k] = z_{i-1}[k] + \sum_{n=0}^3 |t[q+n] - X[r + (1+s)k + P_n]|$$

|         |                                                                        |
|---------|------------------------------------------------------------------------|
| $z_i[]$ | Result accumulator (inoutW) of iteration i                             |
| $X[r]$  | Input slice [inA, inB], starting at position r. in 2D mode X is matrix |
| $t[q]$  | Target vector [inC , inD] , starting at position q                     |
| $P_n$   | 2D position defined by the pattern                                     |
| $s$     | Number of elements to skip                                             |
| $k$     | Index of the element position in the output vector                     |

### 5.7.2 Sliding window, subtract and compare

The basic VSWGGENSUBCMP process of each result element can be generalized to the formula:

$$z_i[k] = z_{i-1}[k] + \sum_{n=0}^3 \begin{cases} src, & \text{if } val < th \\ 0, & \text{if } val \geq th \end{cases} \quad \left| \quad \begin{array}{l} src = X[r + (1+s)k + P_n] \\ val = |c[q+k] - src| \end{array} \right.$$

|         |                                                                        |
|---------|------------------------------------------------------------------------|
| $z_i[]$ | Result accumulator (inoutW) of iteration i                             |
| $X[r]$  | Input slice [inA, inB], starting at position r. in 2D mode X is matrix |
| $c[q]$  | Centroid vector [inC , inD] , starting at position q                   |
| $P_n$   | 2D position defined by the pattern (see below)                         |
| $s$     | Number of elements to skip                                             |
| $k$     | Index of the element position in the output vector                     |

|           |                                                                                       |
|-----------|---------------------------------------------------------------------------------------|
| <i>th</i> | Threshold to compare result to (count the number of elements that pass the threshold) |
|-----------|---------------------------------------------------------------------------------------|

### 5.7.3 Sliding window, subtract and saturate

The basic SWSUB process of each result element can be generalized to the formula:

$$z[k] = \sum_{n=0}^3 \min(th, \max(-th, x[r + k + n] - t[k], th))$$

|             |                                                    |
|-------------|----------------------------------------------------|
| <i>z[ ]</i> | Result vector                                      |
| <i>x[r]</i> | Input slice [inA, inB], starting at position r     |
| <i>t[ ]</i> | Coefficient vector [inC] , starting at position q  |
| <i>sat</i>  | Number of bits for saturation value calculation    |
| <i>th</i>   | $2^{sat} - 1$                                      |
| <i>k</i>    | Index of the element position in the output vector |

## 5.8 Special data access

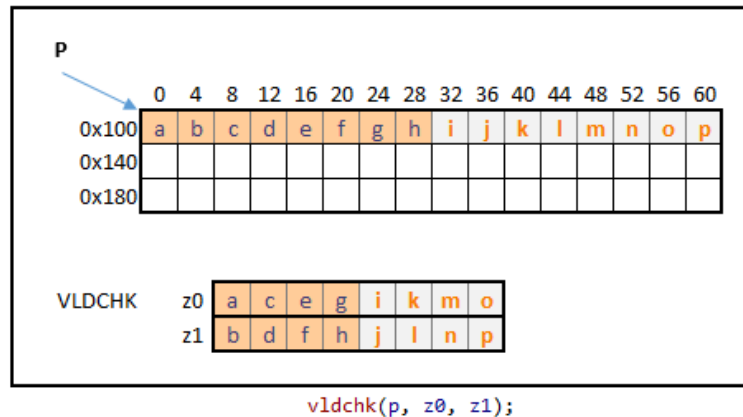
CEVA-XM has several vector load instructions, optimized for memory access in that can be beneficial for various filters and algorithms.

| Instruction   | Units | switches    | Operation                                                                                          | Details                                                                                                             |
|---------------|-------|-------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>vst</b>    | L     | sat<br>satu | $*(n+k) = a[k]$                                                                                    | Store operation of a vector. Can perform signed or unsigned saturation, vector-conditional store and vector unpack. |
| <b>vldchk</b> | LL    |             | $z[k] = * (n + 2k)$<br>$y[k] = * (n + 2k + 1)$                                                     | Checked load (see details below)                                                                                    |
| <b>vldov</b>  | L     |             | $z[k] = * (n + k)$<br>$y[k] = * (n + k + 1)$<br>$u[k] = * (n + 2k + 2)$<br>$v[k] = * (n + 2k + 3)$ | Overlap load (see details below)                                                                                    |

### 5.8.1 Checkered load/store

VLDCHK instruction, reads 512-bit from the memory and permute its content into two output vectors, one contains the even elements and the other the odd elements. This Vec-C instruction uses multiple output vectors

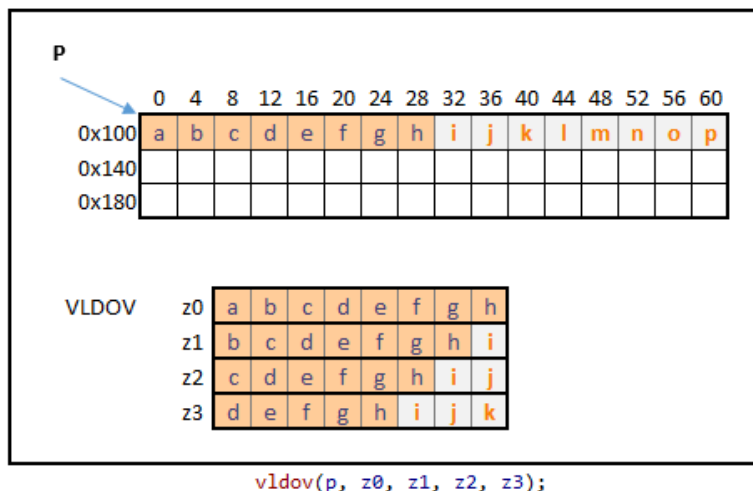
Figure 16: checkered load



### 5.8.2 Overlap load/store

VLDOV instruction reads 512-bit from the memory and permutes the data into 4 vectors, each with different offset from the pointer. This Vec-C instruction uses multiple output vectors

Figure 17: vector load overlap operation



## 5.9 Parallel data access

Parallel memory access is a powerful mechanism of the XM family, allowing access to up-to 32 different addresses in a single cycle, details on parallel data access can be found in chapter [Parallel data access \(protected and unprotected\)](#).

The basic operation of all instructions is the same, the differences are memory access method (bank / bank-coupled / arbitrary) and the way the data is organized in output vectors.

**Table 17: Parallel load/store instructions**

| Instruction     | Units | switches                        | Operation               | Details                                                                                                     |
|-----------------|-------|---------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>vbcpld</b>   | LL    | nonalign<br>fold                | $z[k] = * (m + n[k])$   | Bank Coupled Parallel data load, enables non-aligned element access over 8 virtual banks                    |
| <b>vbcpst</b>   | L     | nonalign<br>sat<br>satu<br>fold | $* (m + n[k]) = a[k]$   | Bank Coupled Parallel data store, enables non-aligned element access over 8 virtual banks                   |
| <b>vpld</b>     | LL    | fold<br>rel                     | $z[k] = * (m + n[k])$   | Parallel Load, please refer to <a href="#">Parallel data access (protected and unprotected)</a>             |
| <b>vpst</b>     | L     | rel<br>sat<br>satu<br>fold      | $* (m + n[k]) = a[k]$   | Parallel Store                                                                                              |
| <b>vldmulti</b> | LL    | x2<br>x4<br>x8                  | $z[k:r] = * (m + n[k])$ | Load half/quarter/eight vector from 2/4/8 different addresses that are evenly spaced in vector <b>n[ ]</b>  |
| <b>vstmulti</b> | L     | x2<br>x4<br>x8                  | $* (m + n[k]) = a[k:r]$ | Store half/quarter/eight vector from 2/4/8 different addresses that are evenly spaced in vector <b>n[ ]</b> |

## 5.10 None-Linear math functions

### 5.10.1 VINV – Vector Inversion

Vector Inversion is a fixed-point implementation of  $\frac{1}{x}$  the output is two fixed-point vectors of 16-bit elements, representing mantissa and exponent for each element. The scalar variant returns a 32-bit register, partitioned of two elements, each 16-bit, the MSB is the exponent and the LSB is the mantissa.

$$\frac{1}{x[k]} = z[k] * 2^{[y[k]-16]}$$

|        |                                                                                                                                                                                               |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| z[ ]   | Result vector : mantissa of the inversion result                                                                                                                                              |
| y[ ]   | Result vector : binary exponent (shiftdev) of the inversion result [0..-15]                                                                                                                   |
| x[ ]   | Input vector of 16-bit unsigned short vector. When the input is a 32-bit unsigned integer, the value is normalized before the inversion and only the 16 MSB are used in the calculation       |
| absexp | A switch that changes the exponent value (shiftdev) into absolute value, easier to use with right-shift instructions. The formula is changed to :<br>$\frac{1}{x[k]} = z[k] * 2^{[-y[k]-16]}$ |

Maximal error = 1.69542E-006

### 5.10.2 VSQRT – Vector Square-Root

Vector square-root is a fixed point implementation of  $\sqrt{x}$  the output is two fixed-point vectors, representing mantissa and exponent for each element. The scalar variant returns a 32-bit register, partitioned of two elements, each 16-bit, the MSB is the exponent and the LSB is the mantissa.

$$\sqrt{x[k]} = z[k] * 2^{[y[k]-8]}$$

|      |                                                                             |
|------|-----------------------------------------------------------------------------|
| z[ ] | Result vector : mantissa of the inversion result                            |
| y[ ] | Result vector : binary exponent (shiftdev) of the inversion result [0..-15] |

|        |                                                                                                                                                                                           |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $x[]$  | Input vector of 16-bit unsigned short vector. When the input is a 32-bit unsigned integer, the value is normalized before the inversion and only the 16 MSB are used in the calculation   |
| absexp | A switch that changes the exponent value (shiftdev) into absolute value, easier to use with right-shift instructions. The formula is changed to :<br>$\sqrt{x[k]} = z[k] * 2^{[-y[k]-8]}$ |

Maximal error = 1.03305E-006

### 5.10.3 VSQRTI – Vector Inverse Square-Root

Vector inverse square root is a fixed point implementation of  $\frac{1}{\sqrt{x}}$  the output is two fixed-point vectors, representing mantissa and exponent for each element. The scalar variant returns a 32-bit register, partitioned of two elements, each 16-bit, the MSB is the exponent and the LSB is the mantissa.

$$\frac{1}{\sqrt{x[k]}} = z[k] * 2^{[y[k]-16]}$$

|        |                                                                                                                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $z[]$  | Result vector : mantissa of the inversion result                                                                                                                                                     |
| $y[]$  | Result vector : binary exponent (shiftdev) of the inversion result [0..15]                                                                                                                           |
| $x[]$  | Input vector of 16-bit unsigned short vector. When the input is a 32-bit unsigned integer, the value is normalized before the inversion and only the 16 MSB are used in the calculation              |
| absexp | A switch that changes the exponent value (shiftdev) into absolute value, easier to use with right-shift instructions. The formula is changed to :<br>$\frac{1}{\sqrt{x[k]}} = z[k] * 2^{[-y[k]-16]}$ |

Maximal error = 7.24802E-006

## 5.10.4 VATAN – Vector Arc-Tangent

Vector inverse-tangent is a fixed point implementation of  $\text{atan}(x)$

$$z[k] = \text{atan}(x[k])$$

|       |                                                                                                                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $z[]$ | Result vector in a fixed point format $\left[-\frac{1}{2} \dots \frac{1}{2}\right]$ Q1.15, representing the range of $\left[-\frac{\pi}{2} \dots \frac{+\pi}{2}\right]$                                           |
| $x[]$ | Input short8 vector in a fixed point format $[-32 \dots 32]$ Q6.10<br>When used with scalar input as 32-bit integer, the value is normalized before the inversion and only the 16 MSB are used in the calculation |

Maximal error = 8.92856934E-5

### 5.10.5 VTANH – Vector Hyperbolic Tangent

This instruction calculates natural logarithm of a *single* element in the vector (does not calculate a vector). The position of the calculated element is set by variable *b*.

$$z[k] = \tanh x[k]$$

|             |                                                      |
|-------------|------------------------------------------------------|
| <i>z</i> [] | Result vector in the range of [-1..+1] Q2.14         |
| <i>a</i> [] | Input vector short16, with vale range [-8..+8] Q4.12 |

Maximal error = 1.21383854682E-4

### 5.10.6 VEXPI – Natural Exponent Inverted

This instruction calculates inverted exponent calculation of a *single* element in the vector (does not calculate a vector). The position of the calculated element is set by variable *b*. If the value of the calculated element is positive it is inverted

$$z[b] = \begin{cases} e^{-a[b]}, & a[b] \geq 0 \\ e^{a[b]}, & a[b] < 0 \end{cases}$$

|                       |                                                               |
|-----------------------|---------------------------------------------------------------|
| <i>z</i> [ <i>b</i> ] | Result vector (only element <i>b</i> ) in range of [0..1] Q16 |
| <i>y</i>              | VPR that marks whether the calculated element was positive    |
| <i>a</i> []           | Input vector short16, with vale range [-8..+8] Q4.12          |

Maximal error = 3.93339246E-5

### 5.10.7 VLOGE – Natural Logarithm

This instruction calculates natural logarithm of a *single* element in the vector (does not calculate a vector). The position of the calculated element is set by variable *b*.

$$z[b] = \log_e(a[b])$$

|                       |                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| <i>z</i> [ <i>b</i> ] | Result vector (only element <i>b</i> ) in a fixed point format Q5.11                                     |
| <i>b</i>              | Element index in vector <i>a</i> [] to calculate                                                         |
| <i>a</i> []           | Input vector ushort16/ uint8. Value is normalized to 17 bit and MSB 16-bits are used for the calculation |

Maximal error = 4.880662183E-4

### 5.10.8 VSIN – Sine and Cosine

This instruction calculates sine or cosine of a *single* element in the vector (does not calculate a vector). The position of the calculated element is set by variable *b* and cosa switch defines cosine calculation

$$z[b] = \begin{cases} \sin a[b], & no\_switch \\ \cos a[b], & cosa \end{cases}$$

|             |                                                                                                      |
|-------------|------------------------------------------------------------------------------------------------------|
| <i>z[b]</i> | Result vector (only element <i>b</i> ) in a fixed point format Q2.14 output range [-1...+1]          |
| <i>b</i>    | Element index in vector <i>a[ ]</i> to calculate                                                     |
| <i>a[ ]</i> | Input vector short16 in fixed point format Q1.15 [-π ... + π] in radians, normalized to [-1 ... + 1] |

Maximal error = 1.1126956008e-4

## 5.11 floating-point

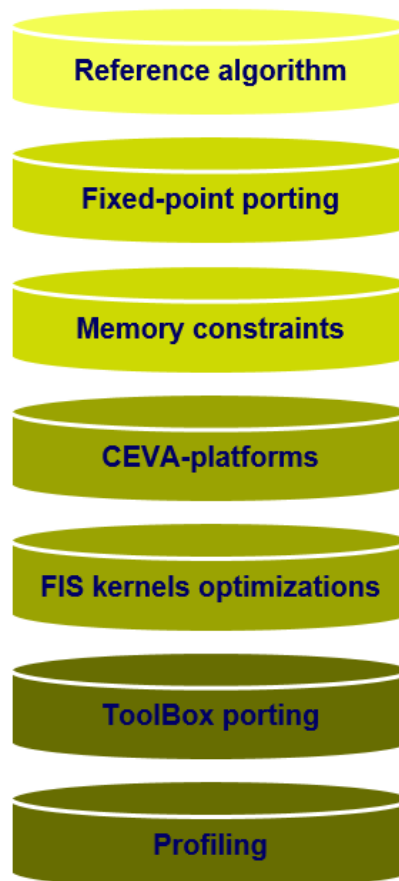
vfp2hfp,vfp2int,vfp2long,vfp2usint, vfp2uslong,vfpadd,vfpcmp,vfpcombine,vfpextract, vfpinv, vfpmac, vfpmax, vfpmin, vfpmpy, vfpmsu, vfpsqrt, vfpsqrti, vfpsub, vusint2fp, vusint2hfp, vuslong2fp, vuslong2hfp, vint2fp, vint2hfp, vhfp2fp, vhfp2int, vhfp2long, vhfp2usint, vhfp2uslong, vhfpadd, vhfpcmp, vhfpcombine, vhfextract, vhfpinv, vhfpmac, vhfpmmax,vhfpmmin, vhfpmmpy, vhfpmmsu, vhfpsqrt,vhfpsqrti, vhfpsub, vlong2fp, vlong2hfp

## 6. Code Optimization Guide

### 6.1 Porting to CEVA-XM work flow

When optimizing an existing algorithm from reference implementation (assume PC) to optimized DSP code, there are several phases that the algorithm must pass before running in optimal manner.

*Figure 18: algorithm porting flow*



## 6.2 Reference Algorithm

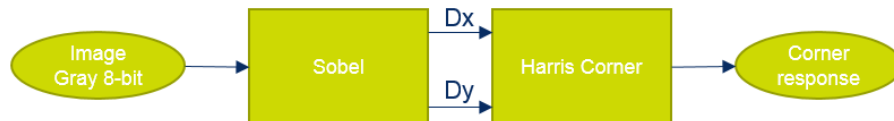
The assumption is that optimization process begins with reference algorithm, preferable with extensive verification platform running on any PC system (windows / Linux). Starting with such validation platform will accompany most of the porting process.

Before starting the porting process, the algorithm should be analyzed and each block should be estimated for its footprint consisting of:

- Interfaces
- Memory usage
- Processing accuracy
- Processing cycles

This breakdown and estimation will focus the work in the following stages.

For our test case let's consider the following algorithm:



```
cornerHarris(src, dst, 3, 3, 0.04);
```

The reference is a floating-point OpenCV implementation. We'll examine the process of implementing this algorithm in CEVA-XM6.

## 6.3 Fixed point porting

Most algorithms are designed in floating-point or other high-precision platform. CEVA-XM6 DSP supports SIMD floating point operations both 32-bit and 16-bit precision, but delivers best performance in fixed-point arithmetic. The first thing we'll examine will be to port any significant parts of the algorithm to fixed-point processing.

Reduction of precision from 32-bit fixed to 16-bit or even to 8-bit accuracy can also be beneficial when striving to best possible performance, as CEVA-XM cores deliver twice the performance in lower precision processing, both in processing power and in memory bandwidth.

Some parts cannot be ported to fixed-point arithmetic and must remain in floating-point accuracy. For these parts of the algorithm, exist the floating-point instructions. Using floating-point operations have the obvious advantage of accuracy and two drawbacks of memory usage (larger size elements) and throughput (8 operations per cycles). When possible, half-precision floating point operations can be a good compromise.

In Our example, Sobel filter can be easily implemented with fixed-point accuracy as it is a 3x3 convolution with fixed-point values

$$Dx \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} Dy \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

**Code 6-1:Sobel reference implementation**

```
short Dx, Dy;
Dx = pix[1-s] + 2 * pix[1] + pix[1+s] -
    (pix[-1-s] + 2 * pix[-1] + pix[-1+s]);
Dy = pix[-1 + s] + 2 * pix[s] + pix[1 + s] -
    (pix[-1 - s] + 2 * pix[-s] + pix[1 - s]);
```

The Harris corner is a bit more complex but can be implemented in fixed point:

**Code 6-2:Harris corener reference implementation**

```
for (k=0;k<3;k++) //accumulating sigma{dxdy}
    for (l=0;l<3;l++)
        s32SigmaDxDy += s16Dx[k][l]*s16Dy[k][l];
s16SigmaDxDy = (short)((s32SigmaDxDy >> 9) & 0xffff);

for (k=0;k<3;k++) //accumulating sigma{dx^2}
    for (l=0;l<3;l++)
        s32SigmaDx2 += s16Dx[k][l]*s16Dx[k][l];
s16SigmaDx2 = (short)((s32SigmaDx2 >> 9) & 0xffff);

for (k=0;k<3;k++) //accumulating sigma{dy^2}
    for (l=0;l<3;l++)
        s32SigmaDy2 += s16Dy[k][l]*s16Dy[k][l];
s16SigmaDy2 = (short)((s32SigmaDy2 >> 9) & 0xffff);

//calculating (sigma(dx^2) + sigma(dy^2)) >> u16k
//s16SigmaDx2pSigmaDy2 = (s16SigmaDx2 + s16SigmaDy2) >> u16k;
s32SigmaDx2pSigmaDy2 = (((s16SigmaDx2 + s16SigmaDy2)*(s16SigmaDx2 + s16SigmaDy2) +
16384) >> 15) * u16k;

//result = sigma(dx^2)*sigma(dy^2)
s32Result = s16SigmaDx2*s16SigmaDy2;
//result -= sigma(dxdy)^2
s32Result -= s16SigmaDxDy*s16SigmaDxDy;
//result -= ((sigma(dx^2) + sigma(dy^2)) >> u16k) >> 8
s32Result -= s32SigmaDx2pSigmaDy2;
s32Result >>= 8;

p_s16Dst[i*s32DstStep + j] = (short)(s32Result & 0xffff);
```

Once the kernels are implemented, the algorithms accuracy should be tested over the complete tests vectors, as from this point forward, only performance will be optimized.

## 6.4 Memory constraints considerations

CEVA-XM DSPs are TCM based cores, thus all data that need to be processed, should reside in the IDM for kernel efficiency reasons both cycles

and power. This phase requires system level understanding, data mapping and DMA usage.

First step is to estimate the amount of memory that need to be used during the performance-critical stages of the algorithm. The following should be taken into consideration:

- Input / output buffers sizes
- Temporary buffers, when needed
- Double or Triple buffers for DMA efficiency
- Look Up Tables (LUT) used by the kernels
- Memory used by the system
  - Drivers
  - OS
  - Standard-Libraries data
  - Program Stack
- Memory block sizes and boundaries

In many cases the input (and output) is 2D data such as image, long lists or complex data structures that will not fit into IDM and there will be need to swap in and out of the IDM, using DMA. For that purpose, a DMA driver is supplied that simplifies the DMA configuration to an intuitive API, more details in CEVA-XM6 iDMA driver user guide

Let's examine our example's considerations for mapping the data into IDM.

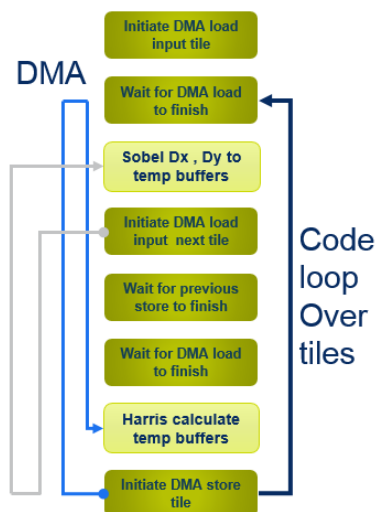
Assumptions:

- Total system IDM is 256kB, each block is 64kB
- OS, Drivers, Stack and data used by system, 20kB
- Input image is 2D 8-bit grayscale of any size
- Output of Harris response is a 2D 16-bit image
- During calculation a temporary buffer is required (Dx, Dy)

Please note that due to the usage of temporary buffers, there is no need to use double-buffers for input and output. It simplifies buffers mapping to various memory blocks, allows maximization of tile sizes and makes the system flow simple as can be seen here:

The system flow design based on these assumptions is:

Figure 19: Harris corner system flow



Our runtime theoretical memory requirements will be as follows:

Table 18: Harris corner theoretical buffers

| Buffer name                   | Size       | Memory Block |
|-------------------------------|------------|--------------|
| <b>System data</b>            | <b>11k</b> | <b>#0</b>    |
| Input image tile buffer       | 35k        | #0           |
| Temporary Buffer Sobel-Dx     | 70k        | #0,#1        |
| Temporary Buffer Sobel-Dy     | 70k        | #1,#2        |
| Output buffer Harris response | 70k        | #2,#3        |

In practice such buffer partition will maximize tile size and memory usage but will possibly cause two problems that will affect the performance due to the fact that buffers cross memory block boundaries:

- DMA / Core possible contentions
  - While output buffer is being stored from bank #2, Sobel kernel will write to bank #2 the Dy buffer
  - While input buffer is being loaded to bank #0, Harris corner kernel will read from bank#0 the Dx buffer
- Core LSU0 / LSU1 possible contentions
  - Sobel kernel will read and write to bank #0
  - Harris kernel will read and write from bank#2

These contentions are statistical and highly influenced by memory latency and kernels efficiency.

In order to avoid these potential contentions, it is recommended to use the following memory mapping:

**Table 19: Harris corner runtime buffers**

| Buffer name                   | Size       | Memory Block |
|-------------------------------|------------|--------------|
| <i>System data</i>            | <i>11k</i> | <i>#0</i>    |
| <i>Free memory</i>            | <i>21k</i> | <i>#0</i>    |
| Input image tile buffer       | 32k        | #0           |
| Temporary Buffer Sobel-Dx     | 64k        | #1           |
| Temporary Buffer Sobel-Dy     | 64k        | #2           |
| Output buffer Harris response | 64k        | #3           |

Although we do not exploit the whole memory space, efficiency is achieved.

CEVA-CV algorithms contain several implementations of similar algorithms implementation.

CEVA-VX libraries supply an automated tool for solving these issues and take into consideration all relevant elements, more on CEVA-VX usage in chapter [Using CEVA-XM software platforms](#).

## 6.5 Multi-Scalar optimization technique

The two modes are very similar and both enable CEVA-XM cores to perform sophisticated SIMD processing.

In both modes, unrelated data patches (usually 2D) are loaded into separate memory data banks and processed in parallel. Such optimization technique is often referred as multi-scalar processing since each data element is a single scalar value as opposed to correlated vector of data. The processing of data is performed on 16 or 8 elements simultaneously.

There are several differences between one-bank and two-bank modes.

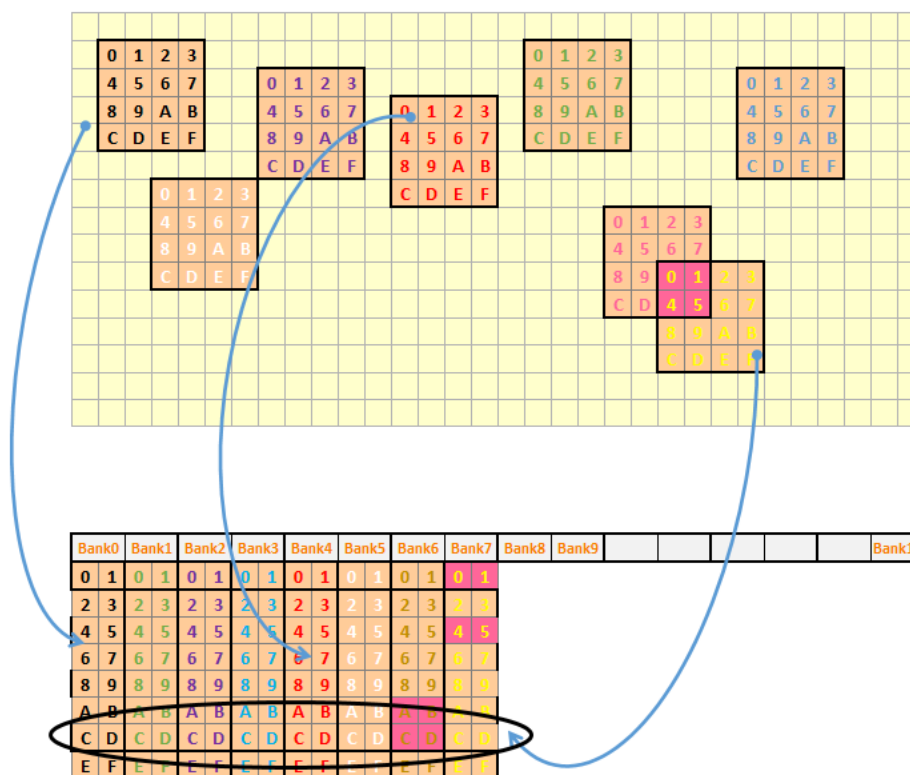
- One-Bank mode data access is restricted by aligned access
- Two-Bank mode enables process of 8 data patches as opposed to 16 in One-Bank mode
- **vpld** / **vpst** instructions are designed for one-bank mode
- **vbcpld** / **vbcpst** instructions are designed for two-bank mode
- When consecutive data access is required, simple linear load/store operations, allow each vector element to contain a data element from a unrelated data patch (see following)

For example filtering of 8 data patches, each 5x5 pixels can be done in the following way:

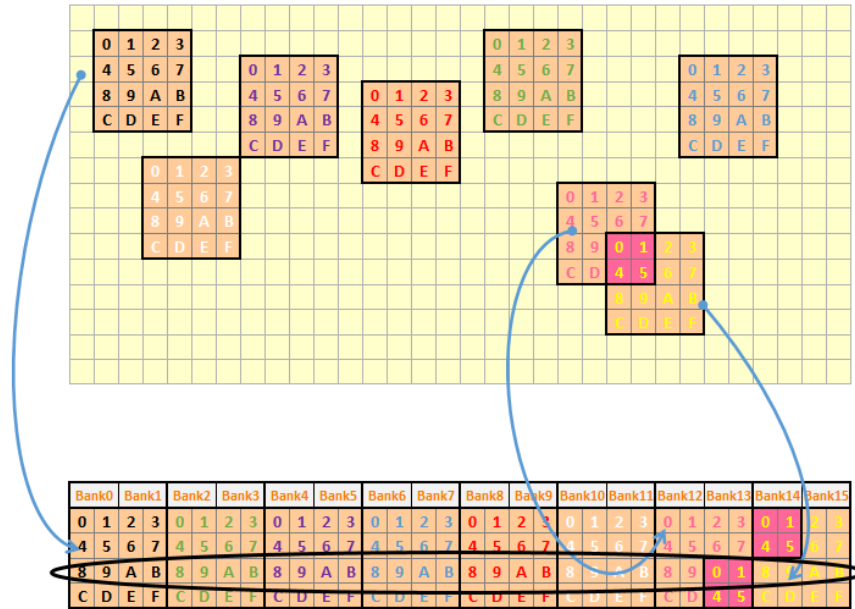
- Each data bank can be loaded with a different patch from external memory, using one-bank DMA transfers
- Each vector load instruction will load a single element from each patch
- The loaded vector can be processed and stored back to the same data structure
- Each patch can be stored back using separate DMA request

The figure below shows the DMA transfers (blue arrow) of 8 4x4 (16-bit) patches from external memory (yellow) into IDM separate data banks. The black circle shows how a simple linear load accesses same element index, from different patches.

**Figure 20: Multi-Scalar memory structure One-Bank**



**Figure 21: Multi-Scalar memory structure Two-Bank**



## 6.6 Writing kernels in FIS

When the system flow and fixed-point implementation is complete and tested, kernel code should be written in Vec-C to achieve best performance.

The most efficient way to write kernels is using FIS library in Microsoft-Visual-Studio or any GCC based development platform.

FIS stand for Functional-Instructions-Simulator and enables us to write functional Vec-C code in the original test environment (MS / GCC) and verify the implementation in the original test environment. Using FIS will allow easy maintenance without the dependency on the core simulation, which is significantly slower than PC.

Writing using FIS is not different than on CEVA-Toolbox IDE.

The following code is an implementation of the Harris corners example kernels:

### Code 6-3:Sobel Vec-C implementation

```
void sobel3x3xdy(uchar *p_u8Src MEM_BLOCK(0),
                short *p_s16DstDx MEM_BLOCK(1),
                short *p_s16DstDy MEM_BLOCK(2),
                int s32SrcStep, int s32DstStepDx, int s32DstStepDy, uint u32Rows, uint u32Cols)
{
    ushort i, j;
    uint u32OutLoop = (u32Cols + 15) >> 4;
    uchar32 v0, v1, v2;
    const uchar* __restrict__ p_in_u8;
    short* __restrict__ p_out_s16dx;
    short* __restrict__ p_out_s16dy;
    short16 v3, v4, v_coeffX, v_coeffY;
    int16 vacc0, vacc1;
    ushort vprRightMask, vprMask;
    uint configSp0A = SWGEN_CTRL0(0, 0, 0, SWGEN_BASE_MODE_2D);
    uint configSp1A = SWGEN_CTRL0(0, 0, 4, SWGEN_BASE_MODE_2D);
    uint configSp0Bx = SWGEN_CTRL1(FILTER_1, 0, SLICE_1, 0xa0);
```

```

uint configSp1Bx = SWGEN_CTRL1(FILTER_1, 0, SLICE_1, 0xaa);
uint configSpBy = SWGEN_CTRL1(FILTER_1, 0, SLICE_1, 0xe0);

short s16KernelX[16] = { -1, 1, 0, 0, -2, 2, -1, 1, 0, 0, 0, 0, 0, 0, 0, 0 };
short s16KernelY[16] = { -1, -2, -1, 0, 1, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 };

vprRightMask = 0xffff;
vprMask = 0xffff;

v_coeffX = *(short16*)s16KernelX;
v_coeffY = *(short16*)s16KernelY;

if (u32OutLoop != u32Cols >> 4)
    vprRightMask = (1 << (u32Cols & 15)) - 1;

for (i = 0; i < u32OutLoop; i++)
{
    p_in_u8 = (uchar *)&p_u8Src[-1 - 1 * s32SrcStep + i * 16];
    p_out_s16dx = (short *)&p_s16DstDx[i * 16];
    p_out_s16dy = (short *)&p_s16DstDy[i * 16];

    if (i == u32OutLoop - 1)
        vprMask = vprRightMask;

    v0 = *(uchar32*)p_in_u8;
    p_in_u8 += s32SrcStep;
    v1 = *(uchar32*)p_in_u8;
    p_in_u8 += s32SrcStep;
    v2 = *(uchar32*)p_in_u8;
    p_in_u8 += s32SrcStep;

    for (j = 0; j < u32Rows; ++j)
    {
        vacc0 = vswgenmac(init, v0, v0, v_coeffX, v_coeffX, configSp0A, configSp0Bx);
        vacc1 = vswgenmac(init, v0, v0, v_coeffY, v_coeffY, configSp0A, configSpBy);
        v0 = v1;
        v1 = v2;
        v2 = *(uchar32*)p_in_u8;
        p_in_u8 += s32SrcStep;

        v3 = vswgenmac(v0, v1, v_coeffX, v_coeffX, configSp1A, configSp1Bx, vacc0);
        v4 = vswgenmac(v1, v1, v_coeffY, v_coeffY, configSp1A, configSpBy, vacc1);

        vst(v3, (short16*)p_out_s16dx, vprMask);
        p_out_s16dx += s32DstStepDx;

        vst(v4, (short16*)p_out_s16dy, vprMask);
        p_out_s16dy += s32DstStepDy;
    }
}
}

```

#### Code 6-4: Harris corner Vec-C implementation

```

void cornerHarris3x3(short *p_s16Dx MEM_BLOCK(0), short *p_s16Dy MEM_BLOCK(1), short *p_s16Dst MEM_BLOCK(2), int
s32SrcStep, int s32DstStep, uint u32Rows, uint u32Cols, ushort u16k)
{
    uint i, j;
    short16 Dx00, Dy00, Dx10, Dy10, Dx20, Dy20;
    short16 Dx01, Dy01, Dx11, Dy11, Dx21, Dy21;
    short16 Dx02, Dy02, Dx12, Dy12, Dx22, Dy22;
    short16 dummy;

    int16 iSigmaDy2;
    uint16 iSigmaDx2;
    int16 iSigmaDxDy, iResult;
    short16 sSigmaDxDy, sSigmaDx2, sSigmaDy2, sSigmaDx2pSigmaDy2, sResult, v_zero;

    v_zero=0;

    uint16 ui_zero = (uint16)0;

    p_s16Dx -= (s32SrcStep + 1); //moving to first load
    p_s16Dy -= (s32SrcStep + 1);

    ushort col_mask = -1;
    ushort last_col_mask = -1;
    if (u32Cols & 15)
        last_col_mask = (1 << (u32Cols & 15)) - 1;

    for (j=0;j<u32Cols;j+=16)
    {
        if (j + 16 >= u32Cols)
            col_mask = last_col_mask;

        for (i=0;i<u32Rows;i++)
            //DSP_CEVA_UNROLL(2)

```

```

{
    /*
    s32Result = 0;
    s32SigmaDxDy = 0;
    s32SigmaDx2 = 0;
    s32SigmaDy2 = 0;
    for (k=0;k<3;k++) { // loading 3x3 neighborhood derivatives
        for (l=0;l<3;l++) {
            s16Dx[k][l] = p_s16Dx[(i + k)*s32SrcStep + (j + 1)];
            s16Dy[k][l] = p_s16Dy[(i + k)*s32SrcStep + (j + 1)];
        }
    }
    */
    vldov((short16*)(p_s16Dx + (i)*s32SrcStep + j), Dx00, Dx01, Dx02, dummy);
    vldov((short16*)(p_s16Dy + (i)*s32SrcStep + j), Dy00, Dy01, Dy02, dummy);
    vldov((short16*)(p_s16Dx + (i + 1)*s32SrcStep + j), Dx10, Dx11, Dx12, dummy);
    vldov((short16*)(p_s16Dy + (i + 1)*s32SrcStep + j), Dy10, Dy11, Dy12, dummy);
    vldov((short16*)(p_s16Dx + (i + 2)*s32SrcStep + j), Dx20, Dx21, Dx22, dummy);
    vldov((short16*)(p_s16Dy + (i + 2)*s32SrcStep + j), Dy20, Dy21, Dy22, dummy);

    /* accumulating sigma{dxdy} and taking absolute value
    for (k=0;k<3;k++) //accumulating sigma{dxdy}
        for (l=0;l<3;l++)
            s32SigmaDxDy += s16Dx[k][l]*s16Dy[k][l];
    s16SigmaDxDy = (short)((s32SigmaDxDy >> 9) & 0xffff); */
    iSigmaDxDy = vmac3(Dx00, Dy00, Dx01, Dy01, (int16)ui_zero);
    iSigmaDxDy = vmac3(Dx02, Dy02, Dx10, Dy10, iSigmaDxDy);
    iSigmaDxDy = vmac3(Dx11, Dy11, Dx12, Dy12, iSigmaDxDy);
    iSigmaDxDy = vmac3(Dx20, Dy20, Dx21, Dy21, iSigmaDxDy);
    sSigmaDxDy = vmac(Dx22, Dy22, 9, iSigmaDxDy);
    //sSigmaDxDy = (short16)vaccshiftr((uchar)9, iSigmaDxDy);
    /* accumulating sigma{dx} and taking absolute value
    for (k=0;k<3;k++) //accumulating sigma{dx^2}
        for (l=0;l<3;l++)
            s32SigmaDx2 += s16Dx[k][l]*s16Dx[k][l];
    s16SigmaDx2 = (short)((s32SigmaDx2 >> 9) & 0xffff); */
    iSigmaDx2 = vsqacc(Dx00, Dx10, Dx01, Dx11, ui_zero);
    iSigmaDx2 = vsqacc(Dx02, Dx12, Dx20, Dx21, iSigmaDx2);
    sSigmaDx2 = vmac(Dx22, Dx22, 9, (int16)iSigmaDx2);
    //sSigmaDx2 = (short16)vaccshiftr((uchar)9, iSigmaDx2);
    /* accumulating sigma{dy} and taking absolute value
    for (k=0;k<3;k++) //accumulating sigma{dy^2}
        for (l=0;l<3;l++)
            s32SigmaDy2 += s16Dy[k][l]*s16Dy[k][l];
    s16SigmaDy2 = (short)((s32SigmaDy2 >> 9) & 0xffff); */
    iSigmaDy2 = vmad(square, init, Dy00, Dy10, Dy10, 0); //3
    iSigmaDy2 = vmac3(square, Dy01, Dy11, iSigmaDy2);
    iSigmaDy2 = vmac3(square, Dy02, Dy12, (int16)iSigmaDy2);
    iSigmaDy2 = vmac3(square, Dy20, Dy21, (int16)iSigmaDy2);
    sSigmaDy2 = vmac(Dy22, Dy22, 9, (int16)iSigmaDy2);
    //sSigmaDy2 = (short16)vaccshiftr((uchar)9, iSigmaDy2);
    // s32SigmaDx2pSigmaDy2 = (((s16SigmaDx2 + s16SigmaDy2) *
    // (s16SigmaDx2 + s16SigmaDy2) + 16384) >> 15) * u16k;
    sSigmaDx2pSigmaDy2 = vadd(sSigmaDx2, sSigmaDy2);
    sSigmaDx2pSigmaDy2 = vmpy(rnd, sSigmaDx2pSigmaDy2, sSigmaDx2pSigmaDy2, 15);
    // s32Result = 0;
    // s32Result -= s16SigmaDxDy * s16SigmaDxDy;
    // s32Result -= s32SigmaDx2pSigmaDy2;
    // s32Result += s16SigmaDx2 * s16SigmaDy2;
    // s32Result >= 8;
    iResult = vsubmpy(v_zero, sSigmaDxDy, sSigmaDxDy);
    short u16k_m = -u16k;
    iResult = vmac(sSigmaDx2pSigmaDy2, u16k_m, iResult);
    sResult = vmac(sSigmaDx2, sSigmaDy2, 8, iResult);
    vst(sResult, (short16*)(p_s16Dst + i*s32DstStep + j), col_mask);
}
}
}

```

## 6.7 Using CEVA-XM software platforms

### 6.7.1 CEVA-CV

The CEVA-CV libraries provide optimized Vec-C implementation of Computer Vision kernels and algorithms for the CEVA-XM6 DSP platform.

The library contains two major parts:

- **CEVA-CV kernels**  
Group of libraries that contain optimized Vec-C kernels that can be used to study and implement custom kernels. The libraries are divided to subjects as follows

**Table 20: CEVA-CV kernels**

| Library           | Description                                                     |
|-------------------|-----------------------------------------------------------------|
| Feature Library   | Contains functions for feature detection and extraction         |
| Format Library    | Contains functions for color space and pixel format conversion  |
| Geometric Library | Contains functions for geometrical transformations of 2D images |
| Image Library     | Contains functions for image filters and conversions            |
| Math Library      | Contains functions for math conversions                         |
| Object Library    | Contains functions for objects operations                       |

- **CEVA-CV algorithms**  
A set of optimized complete algorithms, including custom system code for each algorithm

**Table 21: CEVA-CV algorithms**

| Library                       | Description                                                                                                               |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| BRIEF Descriptor Matcher      | Implements a brute-force matcher for brief descriptors                                                                    |
| Canny                         | Implements a Canny edge detector                                                                                          |
| Connected Component Labeling  | Implements Connected Component Labeling (CCL)                                                                             |
| FAST-9                        | Implements the FAST-9 feature detector                                                                                    |
| Gaussian Pyramid              | Implements a Gaussian pyramid                                                                                             |
| GFTT (Good Features to Track) | Implements the Good Features to Track (GFTT) algorithm                                                                    |
| Gtrans                        | Implements a Geometric Transformation                                                                                     |
| HOG                           | Performs HOG descriptor generation                                                                                        |
| Image Warp Affine             | Performs affine transformation on a frame buffer                                                                          |
| KLT (Kanade-Lucas-Tomasi)     | Implements the Kanade-Lucas-Tomasi (KLT) sparse optical flow                                                              |
| Laplacian Pyramid             | Performs a Laplacian pyramid transformation                                                                               |
| ORB                           | Implements the ORB descriptor/extractor and feature detector                                                              |
| Pyramid And Detect            | Generates a Gaussian pyramid, and then detects features on a specific level of the pyramid using a Harris corner detector |

More details on CEVA-CV can be found

## 6.7.2 CEVA-VX

CEVA-VX software platforms for XM family is an OpenVX standard implementation for CEVA-XM cores.

Using a standard API, defined by khronos the platform automatically generates CEVA-XM optimized system code and uses CEVA-CV kernels to implement the defined algorithm with best performance and minimal effort.

The code below, generates the complete system code and uses CEVA-CV existing kernels to implement the Harris corner example:

**Code 6-5:CEVA-VX example**

```
#include <stdlib.h>
#include <assert.h>
#include <stdio.h>

#include <VX/vx_ceva_api.h>
#include <VX/vx_khronos_kernels.h>
#include <VX/vx_khronos_nodes.h>
#include <VX/vx_ceva_kernels.h>
#include <VX/vx_ceva_nodes.h>

#include "utils.h"

#define IN_PREFIX "../in/"
#define OUT_PREFIX "../out/"

vx_status SobelHarrisScoreExample()
{
    vx_status status = VX_SUCCESS;
    vx_context context = vxCreateContext();
    vx_graph graph = vxCreateGraph(context);

    cvx_kernel_loader_fp kernels[] =
    {
        vxSobel3x3Kernel,
        vxCevaCVHarrisScoreKernel
    };
    cvxLoadKernels(context, kernels, dimof(kernels));

    if (vxGetStatus((vx_reference)context) == VX_SUCCESS)
    {
        vx_uint32 i = 0, w = 640, h = 480;
        vx_uint32 range = 256, windows = 16;
        vx_image images[] = {
            vxCreateImage(context, w, h, VX_DF_IMAGE_U8), /* 0: inputImg */
            vxCreateVirtualImage(graph, w, h, VX_DF_IMAGE_S16), /* 1: Dx */
            vxCreateVirtualImage(graph, w, h, VX_DF_IMAGE_S16), /* 2: Dy */
            vxCreateImage(context, w, h, VX_DF_IMAGE_S16), /* 3: Output */
        };

        if (vxGetStatus((vx_reference)graph) == VX_SUCCESS)
        {
            vx_float32 k = 0.04;
            vx_int32 bSize = 3;
            vx_scalar s_k = vxCreateScalar(context, VX_TYPE_FLOAT32, &k);
            vx_scalar s_bSize = vxCreateScalar(context, VX_TYPE_INT32, &bSize);

            vx_node nodes[] =
            {
                vxSobel3x3Node(graph, images[0], images[1], images[2]),
                vxCevaCVHarrisScoreNode(graph, images[1], images[2], images[3], s_k, s_bSize)
            };
            vx_border_mode_t border = { VX_BORDER_MODE_CONSTANT, 0 };

            status |= vxSetNodeAttribute(nodes[0], VX_NODE_ATTRIBUTE_BORDER_MODE, &border, sizeof(vx_border_mode_t));
            status |= vxSetNodeAttribute(nodes[1], VX_NODE_ATTRIBUTE_BORDER_MODE, &border, sizeof(vx_border_mode_t));
        }
        if (status == VX_SUCCESS)
        {
            status = vxVerifyGraph(graph);
            if (status == VX_SUCCESS)
            {
                status = readRAWImage(in_640x480_400.raw", images[0]);
                if (status == VX_SUCCESS)
                {

```

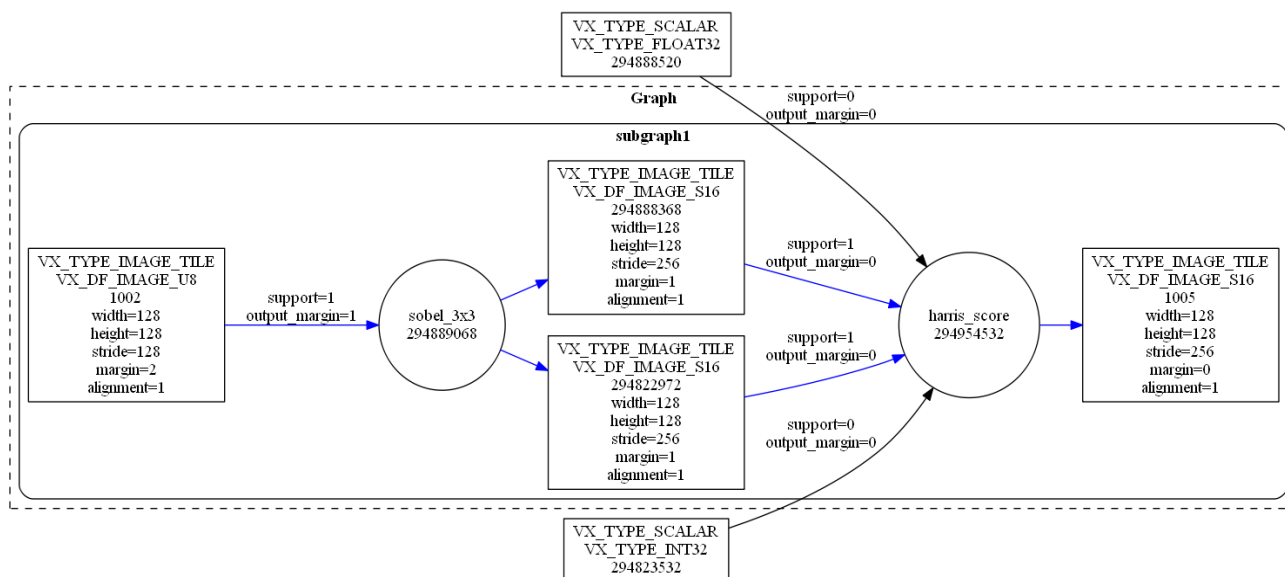
```

status = vxProcessGraph(graph);
if (status == VX_SUCCESS)
{
    status = writeRAWImage(images[3], out_640x480_400.raw");
    if (status == VX_SUCCESS)
        printf("\nHarris Corner graph passed\n");
    else
        printf("\nwriteRAWImage failed\n");
    }
    else
        printf("\nvxProcessGraph failed\n");
    }
    else
        printf("\nreadRAWImage failed\n");
    }
    else
        printf("\nHarris Corner graph failed (%d)\n", status);
    }
    vxReleaseContext(&context);
}
return status;
}

```

The actual system flow, generated from the above OpenVX-Graph is shown below, it is automatically generated and has great resemblance to the manually designed flow.

**Figure 22:CEVA-VX Harris corner flow**



OpenVX standard can be found in Khronos [website](https://www.khronos.org/openvx/)

All the details on CEVA-VX usage can be found in “CEVA-VX Framework Reference guide”

## 7. Optimized Kernels Overview

### 7.1 Mean and Variance

The function `IntensityStats2()` calculates the mean and variance of a 2D patch (with the assumption of normal distribution). The function implements the following formulas:

$$matissa = \frac{2^{exponent}}{N * M}$$

$$mean = \left( \left( \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} src[n, m] \right) * matissa \right) \gg exponent$$

$$variance^2 = \left( \left( \left( \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} src[n, m]^2 \right) * matissa \right) \gg exponent \right) - mean^2$$

Important notes:

- The usage of the directive `__restrict__` to imply the compiler that the input pointer is not aliased, thus it can perform efficient unrolling optimizations
- The instruction `vinv()` is used to calculate the mantissa and exponent for the fixed-point division of the mean and variance.
- Casting is used to convert the loaded pixels, `char32`, into `short32` which is a combination of two vectors. The two vectors are accessed using `vunpack_hi()` and `vunpack_lo()` directives
- `vmac5()` instruction is used to summarize 4 vectors (`short16`) in a single instruction
- `vmac3()` instruction is used with square option to summarize 2 squared vectors (`short16`)
- `vselect()` instruction is used to handle the last tile column when the width is not divisible by 32 by setting the values outside the tile to zero
- `vintrasum()` is used to summarize the accumulators into a single value

## Code 7-1: IntensityStats2 Vec-C function

```

void intensityStats2(uchar * restrict p_u8Src,
                    int s32SrcStride, uint u32Rows, uint u32Cols,
                    uint *p_u32Mean, uint *p_u32Variance)
{
#define MAC_VAL ((1 << 16) + 1)
    uint i, j;
    uint16 AccSigmaX, AccSigmaX2;

    // Pre-calculate division factor 1/N
    int mantissa_exp = vinv((uint)(u32Cols*u32Rows));
    ushort mantissa = (ushort)(mantissa_exp & 0xffff);
    short exponent = (16 - (mantissa_exp >> 16));

    AccSigmaX = 0;
    AccSigmaX2 = 0;

    for (j = 0; j < (u32Cols&(~31)); j += 32)    // Loop over columns, 32 elements every iteration
        for (i = 0; i < (u32Rows&~1); i += 2)    // Loop over rows each iteration works on two rows
        {
            ushort32 V0 = (ushort32)*(uchar32*)&p_u8Src[j + i*s32SrcStride];
            ushort32 V1 = (ushort32)*(uchar32*)&p_u8Src[j + (i + 1)*s32SrcStride];
            AccSigmaX = vmac5(vunpack_hi(V0), unpack_hi(V1), unpack_lo(V0), unpack_lo(V1), MAC_VAL, MAC_VAL, AccSigmaX);
            AccSigmaX2 = vmac3(square, unpack_hi(V0), unpack_lo(V0), AccSigmaX2);
            AccSigmaX2 = vmac3(square, unpack_hi(V1), unpack_lo(V1), AccSigmaX2);
        }
    if (u32Rows & 1)    // Handle the last row in odd patch height
    {
        ushort32 V0 = (ushort32)*(uchar32*)&p_u8Src[j + i*s32SrcStride];
        AccSigmaX = vmac5(vunpack_hi(V0), (ushort16)0, unpack_lo(V0), (ushort16)0, MAC_VAL, MAC_VAL, AccSigmaX);
        AccSigmaX2 = vmac3(square, unpack_hi(V0), unpack_lo(V0), AccSigmaX2);
    }

    if (j < u32Cols)    // Handle last column when patch size is not multiple of 32
    {
        uint vpr_edge = ((1 << (u32Cols - j)) - 1);    // use VPR to mask elements outside the patch
        for (i = 0; i < (u32Rows&~1); i += 2)    {
            uchar32 v0 = *(uchar32*)&p_u8Src[j + i*s32SrcStride];
            uchar32 v1 = *(uchar32*)&p_u8Src[j + (i + 1)*s32SrcStride];
            ushort32 V0 = (ushort32)vselect(v0, (uchar32)0, vpr_edge);
            ushort32 V1 = (ushort32)vselect(v1, (uchar32)0, vpr_edge);
            AccSigmaX = vmac5(vunpack_hi(V0), unpack_hi(V1), unpack_lo(V0), unpack_lo(V1), MAC_VAL, MAC_VAL, AccSigmaX);
            AccSigmaX2 = vmac3(square, unpack_hi(V0), unpack_lo(V0), AccSigmaX2);
            AccSigmaX2 = vmac3(square, unpack_hi(V1), unpack_lo(V1), AccSigmaX2);
        }
    }
    if (u32Rows & 1)    // Handle the last row in odd patch height
    {
        uchar32 v0 = *(uchar32*)&p_u8Src[j + i*s32SrcStride];
        ushort32 V0 = (ushort32)vselect(v0, (uchar32)0, vpr_edge);
        AccSigmaX = vmac5(vunpack_hi(V0), (ushort16)0, unpack_lo(V0), (ushort16)0, MAC_VAL, MAC_VAL, AccSigmaX);
        AccSigmaX2 = vmac3(square, unpack_hi(V0), unpack_lo(V0), AccSigmaX2);
    }
}

// Summarize the results
uint u32SigmaX, u32SigmaX2;
u32SigmaX = vintrasum(vunpack_lo(AccSigmaX));
u32SigmaX += vintrasum(vunpack_hi(AccSigmaX));
u32SigmaX2 = vintrasum(vunpack_lo(AccSigmaX2));
u32SigmaX2 += vintrasum(vunpack_hi(AccSigmaX2));

//compute mean & square-variance
uint u32MeanTmp = (((long long)mantissa*(long long)u32SigmaX) >> exponent);
*p_u32Mean = u32MeanTmp;
//var(x) = E(x^2) - (Ex)^2
*p_u32Variance = (((long long)mantissa*(long long)u32SigmaX2) >> exponent) - (u32MeanTmp*u32MeanTmp);
}

```

## Code 7-2: The resulted assembly code (main loop only)

```

BB_intensityStats2:
; <loop> unrolled 4 times
VPU0.vmac3{ square } v9.us16, v6.us16, v3.ui8, v4.ui8
|| LS0.vld(r24.ui).i8 + r30, v31.i8
|| VPU1.vcast{ twooutput } v11.uc32, v29.us16, v27.us16
|| LS1.modr(r26.ui).i + #8
VPU0.vmac3{ square } v15.us16, v14.us16, v3.ui8, v4.ui8
|| LS0.vld(r7.ui).i8 + r30, v30.i8
|| VPU1.vcast{ twooutput } v13.uc32, v23.us16, v21.us16
|| LS1.vld(r24.ui).i8 + r30, v25.i8
|| LVPU.vcast{ twooutput } v5.uc32, v24.us16, v22.us16

```

```

VPU0.vmac3{ square } v7.us16, v10.us16, v3.ui8, v4.ui8
|| LS0.vld(r7.ui).i8 + r30, v2.i8
|| LS1.vld(r24.ui).i8 + r30, v8.i8
|| VPU1.vmac5 v9.us16, v15.us16, v6.us16, v14.us16, r27.ui, r27.ui, v0.ui8, v1.ui8
VPU0.vmac3{ square } v12.us16, v16.us16, v3.ui8, v4.ui8
|| LVPU.vcast{ twooutput } v31.uc32, v6.us16, v9.us16
|| LS0.vld(r7.ui).i8 + r30, v11.i8
|| VPU1.vmac5 v7.us16, v12.us16, v10.us16, v16.us16, r27.ui, r27.ui, v0.ui8, v1.ui8
VPU0.vmac3{ square } v26.us16, v28.us16, v3.ui8, v4.ui8
|| LVPU.vcast{ twooutput } v30.uc32, v14.us16, v15.us16
|| LS0.vld(r7.ui).i8 + r30, v5.i8
|| LS1.vld(r24.ui).i8 + r30, v13.i8
|| VPU1.vmac5 v26.us16, v27.us16, v28.us16, v29.us16, r27.ui, r27.ui, v0.ui8, v1.ui8
PCU.8.brrcmplp{ ge, calc2, ds2 } r25.i, #-1, #1, #BB_intensityStats2
|| VPU0.vmac3{ square } v27.us16, v29.us16, v3.ui8, v4.ui8
|| LVPU.vcast{ twooutput } v25.uc32, v10.us16, v7.us16
|| VPU1.vmac5 v21.us16, v22.us16, v23.us16, v24.us16, r27.ui, r27.ui, v0.ui8, v1.ui8
VPU0.vmac3{ square } v21.us16, v23.us16, v3.ui8, v4.ui8
|| VPU1.vcast{ twooutput } v8.uc32, v28.us16, v26.us16
|| LVPU.vcast{ twooutput } v2.uc32, v16.us16, v12.us16
VPU0.vmac3{ square } v22.us16, v24.us16, v3.ui8, v4.ui8
|| VPU1.vcast{ twooutput } v11.uc32, v20.us16, v17.us16
|| LVPU.vcast{ twooutput } v8.uc32, v19.us16, v18.us16

```

## 7.2 Separable Correlation 7x7

Separable correlation is constructed of two functions. The first function performs a 1x7 (horizontal flow) correlation into a temporary buffer (keeping the temporary result in high accuracy). The second function performs a 7x1 (vertical flow) correlation into the output buffer.

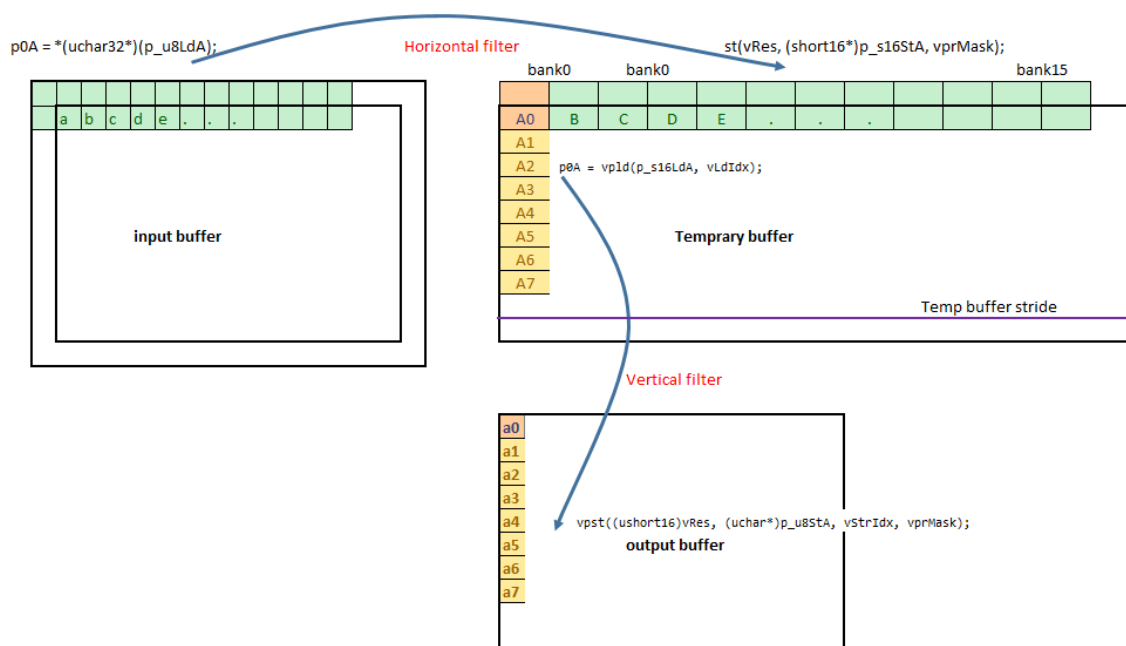
**Code 7-3:Seperable7x7 correlation envelope**

```
void corrSep7x7(uchar *p_u8Src, short *p_s16Tmp, uchar *p_u8Dst, int s32SrcStep, int s32DstStep, short
*p_s16KernelHor, short *p_s16KernelVer, uint u32Rows, uint u32Cols) {

    p_u8Src = p_u8Src - 3 - 3 * s32SrcStep;
    int s32TmpStep = (((((u32Cols)+31) >> 5) << 5) + 2); //Set the temporary buffer stride top avoid bank conflicts

    corrSep1x7Hor(p_u8Src, p_s16Tmp, s32SrcStep, s32TmpStep, u32Rows + 6, u32Cols, p_s16KernelHor);
    corrSep1x7Ver(p_s16Tmp, p_u8Dst, s32TmpStep, s32DstStep, u32Rows, u32Cols, p_s16KernelVer);
}
```

**Figure 23: The following diagram shows the data flow**



Temporary buffer is of 16-bit elements and its stride set by the two factors:

- Number of columns in the input tile
- Memory internal structure width (16-banks x 32-bit = 64 byte)

The stride is set to be the largest multiplication of 32 (short) + 2 (short), setting the stride to be

$$stride = 64k + 4 [bytes]$$

Thus enabling the usage of vertical load / store operations with `vpld()` / `vpst()` instructions, as used by the kernels.

#### Code 7-4: Seperable 1x7 (horizontal) correlation kernel

```
void corrSep1x7Hor(uchar* RESTRICT p_u8Src, short* RESTRICT p_s16Dst,
                  int s32SrcStep, int s32DstStep,
                  uint u32Rows, uint u32Cols, short *p_s16Kernel)
{
    uint n, m;
    short16 vcoeff = *(short16*)p_s16Kernel; // Coeffs
    ushort vprMask = 0xFFFF; // Masks
    uchar *p_in_u8 = p_u8Src;
    short *p_out_s16 = p_s16Dst;
    uchar *p_u8LdA;
    short *p_s16StA;
    uint ctrl_com = SWGEN_CTRL1(FILTER_1, 0, 0, SWGEN_LINE); // Sliding window control word

    ushort vprRightMaskM = 0xFFFF;
    if ((u32Cols & 15) != 0)
        vprRightMaskM = ((1 << (u32Cols & 15)) - 1);

    uint u32FixedM = (((u32Cols + 15) >> 4) << 4); // round up

    for (m = 0; m < u32FixedM; m += 16) {
        p_u8LdA = p_in_u8 + m;
        p_s16StA = p_out_s16 + m;
        if (u32Cols - m < 16) // mask
            vprMask = vprRightMaskM;

        for (n = 0; n < u32Rows; n++) DSP_CEVA_UNROLL(8) {
            uchar32 p0A = *(uchar32*)(p_u8LdA); p_u8LdA += s32SrcStep; // load
            // process
            int16 vacc = vswgenmac(init, p0A, p0A, vcoeff, vcoeff, SWGEN_CTRL0(0, 0, 0, SWGEN_BASE_MODE_1D), ctrl_com);
            short16 vRes = vswgenmac(p0A, p0A, p0A, vcoeff, vcoeff, SWGEN_CTRL0(0, 4, 4, SWGEN_BASE_MODE_1D), ctrl_com, vacc);
            vst(vRes, (short16*)p_s16StA, vprMask); p_s16StA += s32DstStep; // store
        } // end rows
    } // end cols
}
```

#### Code 7-5: Seperable 7x1 (vertical) correlation kernel

```
void corrSep1x7Ver(short* RESTRICT p_s16Src, uchar* RESTRICT p_u8Dst,
                  int s32SrcStep, int s32DstStep,
                  uint u32Rows, uint u32Cols, short *p_s16Kernel)
{
    short16 Idx = vpack(vsequence((unsigned short)1), vsequence((unsigned short)1) + 8); // Indices for vpst
    short16 vStrIdx = Idx*s32DstStep;
    short16 vLdIdx = Idx*s32SrcStep;
    uint n, m;
    short16 vcoeff = *(short16*)p_s16Kernel; // Coefficients
    ushort vprMask = 0xFFFF; // Masks
    short *p_in_s16 = p_s16Src;
    uchar *p_out_u8 = p_u8Dst;
    short *p_s16LdA;
    uchar *p_u8StA;
    uint ctrl_com = SWGEN_CTRL1(FILTER_1, 0, 0, SWGEN_LINE);
    uint ctrl_last = SWGEN_CTRL1(FILTER_1, 0, 0, 0xE0);

    ushort vprRightMaskN = 0xFFFF;
    if ((u32Rows & 15) != 0)
        vprRightMaskN = ((1 << (u32Rows & 15)) - 1);

    uint u32FixedN = (((u32Rows + 15) >> 4) << 4); // round up
    for (n = 0; n < u32FixedN; n += 16) {
        p_s16LdA = p_in_s16 + n*s32SrcStep; p_s16LdB = ;
        p_u8StA = p_out_u8 + n*s32DstStep;
        if (u32Rows - n < 16)
            vprMask = vprRightMaskN;

        for (m = 0; m < u32Cols; m++) // rows
            DSP_CEVA_UNROLL(8)
            {
                int16 vacc;
                short16 vRes;
                short16 p0A = vpld(p_s16LdA, vLdIdx);
                short16 p0B = vpld(p_s16LdA + 16*s32SrcStep, vLdIdx); p_s16LdA++;
                // process
                vacc = vswgenmac(init, p0A, p0B, vcoeff, vcoeff, SWGEN_CTRL0(0, 0, 0, SWGEN_BASE_MODE_1D), ctrl_com);
                vRes = vswgenmac(p0A, p0B, p0B, vcoeff, vcoeff, SWGEN_CTRL0(0, 4, 4, SWGEN_BASE_MODE_1D), ctrl_last, vacc);

                vpst((ushort16)vRes, (uchar*)p_u8StA, vStrIdx, vprMask); p_u8StA++; // Parallel store column
            } // end cols
    } // end row
}
```



